サーバー側（バッ

フロントエンド

ボタンを押したと

し、どうや

- [redacted] — MIDDLEWARE・CORS・SESSION・REST_FRAMEWORK などプロジェクト全体の設
- の基礎

---

0. 前　　識：バックエンドとは何者か
1. リクエストの　— ボタン1つから応　まで
2. プロジェクトとアプリ — hozen_another と kosu
3. モデル・ビュー・シリアライザ・URLの4
4. HTTPメソッドとステータスコード
5. 実コード全解説：member_views.py 全225行
6. 実コード全解説：serializers.py
7. 実コード全解説：urls.py（　線　）
8. 実コード全解説：CustomPagination（main_utils.py）
9. 実コード全解説：signals.py（自　の　更　　）
10. 実コード全解説：middleware と apps.py
11. 実コード全解説：asynchronous_views.py（　　期処　）
12. 実コード全解説：settings.py（プロジェクト設　）
13. セッション　証とCSRF
14. トラブルシューティング
15. 演習
16. この章のまとめ

「この章のコードが何をしているのか」を　解するために、まず　　という言　の意　から押さえます。

ユーザーの　の前（front）で　く
。　アプリでは React で書かれた画　（ボタン・　・　　）。ブラウザの　で　きます。

画　の　（back）で　く　　。　アプリで
は Django（ジャンゴ）というPythonの仕組みで書かれた「サーバー側のプログラム」。ユーザーの　には見えません。データベースに近いところで、データの　存・取　・計算・チェックを　　します。

レストランでたとえると：

お客さん	する人	ブラウザを使う従業員
ウェイター（ホール）	を　き、　　を運ぶ	フロントエンド（React）
厨　（キッチン）	実際に　　を作る	バックエンド（Django）
蔵庫・食材	材　の　管庫	データベース（PostgreSQL）

お客さん（ブラウザ）は厨　（Django）に　接　りません。　ずウェイター（React）を　して
（リクエスト）を出し、　　（データ）を受け取ります。

画　の見た　（フロント）と、データの管　（バック）を　けておくと、
を　しても、もう　　を壊しにくいからです。たとえば「ボタンの色を　える」のはフロントだけの仕事。「　存できる人員の上限を　える」のはバックだけの仕事、というふうに　割が　確になります。

Python言語で「Webサーバー側のプログラム」を作るための　具
一式（フレームワーク）。データベースの操作・URLの振り　け・セキュリティなど、Webアプ

リに　　な　　があらかじめ　っています。「自　でゼロから作らなくていい家具　き　件」のようなものです。

Django REST Framework（ジャンゴ・レスト・フレームワーク）の　。Django に「　　」を作る機　を足す　加　　。　アプリのバックエンドは、画　そのものを　すのではなく、　　として作られています。その主　がこのDRFです。

プログラム　士が会　するための「窓口」。　アプリでは「`/api/member_list/`にアクセスすると人員一　データがってくる」といった、URLごとの窓口の集まりがAPIです。人間向けの画　ではなく、　　だと考えてください。

JavaScript Object Notation の　。データを　字　で　す共　フォーマット。`{"employee_no": 123, "name": "　　"}`のように「　前と　のペア」で書きます。フロントとバックは、このJSONという「共　語」でデータをやり取りします。　4章・　5章でも　場しました。

　章の　図として、まず「リクエストが　る　」全体を　　します。これを　に　れておくと、後で個々のコードを　むとき「あ、これはあの　階の　だな」と位　づけられます。

ブラウザがサーバーに送る「お願い」。「人員一　をください」「この人員を　存してください」などの依　。

サーバーがリクエストに対して　す「　事」。データ　体（JSON）と、成　を　す　号（ステータスコード）が　っています。

```
①                                React    GET /api/member_list/
        ↓
② settings.py    MIDDLEWARE
     CORS         →               → CSRF         →                          …
        ↓
③ urls.py  URL  member_list/                    MemberList
        ↓
④ member_views.py    MemberList.get()
        ↓
⑤
        ↓
⑥ models  member
        ↓
⑦ CustomPagination
        ↓
⑧ serializers.py     member                  JSON
        ↓
⑨ Response  JSON                 200
        ↓
⑩ MIDDLEWARE
        ↓
⑪ React    JSON
```

このうち、　　が「サーバー側（この章の　囲）」です。1つずつコードで確　していきます。

> 　　存・更新・削除の場合は、　の前後で　　が割り込み、「　が・どのデータを・どう　えたか」を自　で `History`（　）テーブルに記　します。これは§9で詳しく見ます。

Django には「プロジェクト」と「アプリ」という2つの　位があります。最初に　ず混　するポイントなので、ここで整　します。

> アプリ全体をまとめる「 」。設 ファイルや、サイト全体の 口を持ちます。 アプリでは `hozen_another/` フォルダがそれ。`settings.py`（全体設 ）や `wsgi.py`（サーバー起 口）が っています。

> 1つの機 のまとまり。 アプリでは `kosu/` フォルダがそれ。モデル・ビュー・URL・シグナルなど、工数管 という機 の 身が全 ここにあります。

たとえると：

プロジェクト	会社全体・ 社ビル	`hozen_another/`
アプリ	署（事業 ）	`kosu/`

アプリはアプリが `kosu` 1つだけのシンプルな構成です。 きなサービスだと、1つのプロジェクトに「ブログアプリ」「ショップアプリ」のように 数のアプリが ることもあります。

```
kosu/                              ←
├── models.py                      ←                8
├── urls.py                        ← URL               §7
├── apps.py                        ←                §10
├── signals.py                     ←    /                §9
├── tasks.py                       ←
├── views/                         ←
│   ├── member_views.py            ←         API  §5
│   ├── serializers.py             ←         JSON       §6
│   ├── asynchronous_views.py      ←           API  §11
│   └──   kosu_views, team_views ...   ←
├── utils/
│   └── main_utils.py              ←           CustomPagination  §8
└── middleware/
    └── clear_session_middleware.py    ←                      §10
```

の小さなアプリでは `views.py` という1ファイルですが、　アプリは機　が多いので、`views/` フォルダの　に機　ごとのファイル（member, kosu, team…）に　けています。　身は　じ「ビュー」です。

　アプリのバックエンドは、ざっくり4つの　者が　携して　いています。最初にこの「　割　　」を　に　れておくと、コードがぐっと　みやすくなります。

	`urls.py`	どのURLが　たら　に　せるか決める	受　・案　係
	`member_views.py`	実際の処　を行う	（指揮を執る）
	`models.py`	データベースとのやり取り	食材　・在庫管
	`serializers.py`	データ⇔JSON の	盛り　け・　膳の係

Model（モデル）・View（ビュー）・Template（テンプレート）の　　字。Djangoの基　構成。　アプリはテンプレート（HTML生成）の　わりにAPIを　すので、Templateの位　に　　が　る、と考えると　かりやすいです。

```
URL  urls.py
  ↳  member_new/                MemberNew
        ↓
        member_views.py    MemberNew.post
  ↳
        ↓
              serializers.py    MemberSerializer
  ↳           JSON    member
        ↓
        models.py    member
  ↳
```

> シリアライズ＝「Pythonのオブジェクト → JSON　字　」への　換（データを送り出すとき）。デシリアライズ＝「JSON → Pythonオブジェクト」への逆　換（データを受け取るとき）。　アプリの `MemberSerializer` は、この　　向の　　を1つのクラスでこなします。

それでは、いよいよ実コードに　ります。まず HTTP の基　　語（§4）を押さえ、その後この章の主　`member_views.py` を1行ずつ　んでいきましょう。

コードを　む前に、何　も出てくる「HTTPメソッド」と「ステータスコード」を先に整　しておきます。

> リクエストの「　詞」。「何をしたいのか」をします。　アプリで使う主なものは次の4つ。

	取　（　む）	Read	人員一　を　示 `MemberList.get`

	作成（送って作る）	Create	人員を新規　　`MemberNew.post`
	更新（　き換える）	Update	人員データを　集 `MemberUpdate.put`
	削除（消す）	Delete	人員を削除 `MemberDelete.delete`

> 　　Create（作る）・Read（　む）・Update（更新する）・Delete（消す）の　　字。データを扱うアプリの基　4　作。`member_views.py` はこの4つがそのままクラスになっています。

> 　　じURLでも「　みたいのか・作りたいのか・消したいのか」で作を　えたいからです。DRFは、　たメソッドに応じて自　で `get` / `post` / `put` / `delete` メソッドを呼び　けてくれます。プログラマは該　メソッドを書くだけでよいのです。

> 　　応　に　く3桁の　号。「うまくいったか・ダメだったか・なぜダメか」を　します。

	OK	成功	更新成功・一　取　成功
	Created	新規作成成功	人員の新規　　成功
	Accepted	受　・処	期タスクがまだ処
	No Content	成功・　身なし	人員の削除成功
	Bad Request	送り手の　　ミス	号重　・検査　合格
	Unauthorized	証（ログインして）	ログイン
	Forbidden	権限　足	権限なしユーザー
	Not Found	見つからない	対象データなし
	Server Error	サーバー側の故障	プログラムの　期せぬエラー

- = 成功（やった！）
- = 送り手（クライアント）の （あなたのミス）
- = 受け手（サーバー）の （こちらの故障）

このセクションが 章の心臓 です。`member_views.py` は を するAPIで、 アプリのCRUDの 型 です。ここを完全に 解すれば、他のビュー（kosu, team, def…）も じパターンなので一気に めるようになります。

このファイルが使う「外 の 具」を みます。データベース操作の ・DRFのAPI ・このアプリ自身のモデルやシリアライザを、上から順 に取り寄せています。

```
#                          4
from django.db.models import Case, When, Value, IntegerField
#             member
from ..models import member
# DRF  API
from rest_framework.views import APIView
# API                Response
from rest_framework.response import Response
# 200  404
from rest_framework import status
# member  JSON
from .serializers import MemberSerializer
#
from ..utils.main_utils import CustomPagination
```

1行ずつ意　を確　します。

- `from django.db.models import Case, When, Value, IntegerField`
  - `django.db.models` は「データベース操作の　具　」。そこから4つの　　を取り出しています。
  - `Case`・`When`・`Value` は　　を、データベースの　い合わせの　で作るための　　です（後の§5.5で「異　・　社」を最後に　べるために使います）。
  - `IntegerField`（整数フィールド）は「その結果は整数だよ」とデータ型を指　する　　です。

> 「　み込む・取り寄せる」という意　。`from A import B` は「Aという場所からBという　具を取り寄せる」。　でいえば「　蔵庫（A）から　（B）を取り出す」イメージ。これを書かないと、その　具は使えません。

- `from ..models import member`
  - `..` は「1つ上のフォルダ」を　します。`member_views.py` は `kosu/views/` の　にあるので、`..` は `kosu/` を指します。つまり `kosu/models.py` から `member` を　み込んでいます。
  - `member` は人員1人を　すデータの設計図（モデル）。従業員　号・　前・所属・休憩時間などの　を持ちます（　身は　8章）。

> ⚠️　　`.` は「　じフォルダ」、`..` は「1つ上のフォルダ」。`from .serializers`（6行　）は　じ `views/` フォルダの `serializers.py`、`from ..models`（2行　）は1つ上の `kosu/models.py`。間違えると `ImportError`（　み込み失　）になります。

- `from rest_framework.views import APIView`
  - DRFが　意した `APIView`（エーピーアイビュー）という　　のクラス。これを継承（後述）すると、`get`・`post` などのメソッドを書くだけでAPIが作れます。
- `from rest_framework.response import Response`
  - `Response`（レスポンス）は「APIの　事」を作るための　　。`Response({...}, status=...)` の形で、データと　号をセットで　します。
- `from rest_framework import status`

- `status`（ステータス）は `HTTP_200_OK`・`HTTP_404_NOT_FOUND` のような　　。数字の `200` を　接書くより `status.HTTP_200_OK` と書くほうが意　が　かりやすいので使います。

- `from .serializers import MemberSerializer`
  - 　じ `views/` フォルダにある `serializers.py` から `MemberSerializer` を　み込みます。これがデータ⇔JSONの　　係（§6で詳説）。
- `from ..utils.main_utils import CustomPagination`
  - `kosu/utils/main_utils.py` から `CustomPagination`（カスタムページネーション）を　み込みます。一　を「20件ずつ」のようにページ　ける　　（§8で詳説）。

> 　　クラスは「　じ性質を持つものの設計図」。`class MemberList(APIView):` は「`APIView` という親の設計図を引き継いで（継承して）、`MemberList` という子を作る」という意　。親が持つ機　（リクエストの受け取り　など）はそのまま使え、足りない　　（実際の処　）だけ自　で書けばよい、というのが継承の　　さです。家業を継ぐイメージ。

> 　　「人員一　」を　すAPIの　体を　義します。`get` メソッドは、ブラウザが「一　をください（GET）」と言ってきたときに　く処　です。最初にセッション（ログイン情　の　管庫）から、ログイン　の従業員　号と、使う工数区　義のバージョンを取り出します。

```
#
# APIView                    API
class MemberList(APIView):
  # GET
  # GET
  def get(self, request):
    #
    #
    login_no = request.session.get('login_No')
    #
    def_ver = request.session.get('input_def')
```

- `class MemberList(APIView):`
  - `APIView` を継承した `MemberList` クラスを宣言。このクラス1つで「人員一　」というAPIの窓口になります。
- `def get(self, request):`　　,
  - `def`（デフ）は「関数（処　のかたまり）を　義する」キーワード。
  - メソッド　が `get` なので、これは　　（≒「データをください」という　み取りのお願い）が　たときに自　で呼ばれます。DRFが「GETならgetメソッドを呼ぶ」と振り　けてくれます。
  - `self`（セルフ）は「このクラス自身」を指す決まりの　1引数（とりあえず「おまじない」と思ってよい）。
  - `request`（リクエスト）には、ブラウザから　いたお願いの全情　（　が・何を・どんな帯情　で）が　っています。

> 　　クラスの　に書かれた関数のこと。

ブラウザとサーバーの「会　の記憶」。HTTPは「1回ごとに　れる」性質（　回はじめまして）なので、ログイン状　を覚えておくためにセッションを使います。ログイン時に「この人は従業員　号123　」とサーバー側に記　し、ブラウザには「あなたの記憶の　（クッキー）」だけを　します。次回以降、　を見せれば「ああ123　さんね」と思い出せる、という仕組み。　アプリでは `SESSION_ENGINE` でデータベースに記憶しています（§12）。

- `def_ver = request.session.get('input_def')`
  - に、セッションから「使　の工数区　義バージョン」を取り出して `def_ver` に　れます。

この一　を見せてよい相手かを順　に確　します。　ログインしていない人を　く　義バージョンが　い人を　く　その従業員に「管　権限」があるか確　し、なければ　く　そもそも人員データが見つからなければ　く。　があれば、その場で　切なエラー　号を　けて　し、処　を　します。

```
    #
    #
    if not login_no:
      # 401
      return Response({'status': 'error', 'message': '
    '}, status=status.HTTP_401_UNAUTHORIZED)
    #
    if not def_ver:
      # 401
      return Response({'status': 'error', 'message': '
          '}, status=status.HTTP_401_UNAUTHORIZED)

    try:
      #
      #                                    1
      member_data = member.objects.get(employee_no=login_no)
      #                 Menu
      #
      if not member_data.authority:
        # 403
        return Response({'status': 'error', 'message': '                          '},
status=status.HTTP_403_FORBIDDEN)
    #
    except member.DoesNotExist:
      #
      # 401
      return Response({'status': 'error', 'message': '                          '},
status=status.HTTP_401_UNAUTHORIZED)
```

- `if not login_no:`
  - `not login_no` は「`login_no` が空（None や空　字）なら True」。つまり「　ログインなら」という条件。
- `return Response({...}, status=status.HTTP_401_UNAUTHORIZED)`
  - `return`（リターン）でその場で処　を打ち切り、　事を　します。これより下の行は実行されません。
  - 　すデータは辞書 `{'status': 'error', 'message': '...'}`（後でJSONになる）。
  - `status=status.HTTP_401_UNAUTHORIZED` は　　。「あなたが　か　からない＝ログインして」という意　の　号です。

> `{'　':　,'　':　}` の形で「　札と　のペア」を　べたPythonのデータ型。JSONとそっくりで、DRFはこれを自　でJSONに　換しま

> す。

- じ形で、 義バージョンが ければ401で します。
- `try:`
  - `try`（トライ）は「以下を試してみる。 でエラーが起きたら、対応する `except` に ぶ」という構 。データベース い合わせは「見つからない」エラーが起きうるので、安全に囲みます。

> 「やってみて（try）、ダメだったら（except）こうする」という 外処 。 でいえば「 を割ってみて（try）、もし っていたら（except） の を使う」。エラーでプログラム全体が止まるのを ぎます。

- `member_data = member.objects.get(employee_no=login_no)`
  - `member.objects` は「memberテーブルへの 口」（マネージャと呼ぶ）。
  - `.get(employee_no=login_no)` は「従業員 号が `login_no` と一 する人員を 取ってくる」。
  - 見つかれば、その人員データが `member_data` に ります。見つからなければ `member.DoesNotExist` というエラーが 生し、32行 の `except` に びます。

> ORM（Object-Relational Mapping）は「Pythonの書き で、データベースを操作できる仕組み」。`member.objects.get(...)` のように書くと、Djangoが で `SELECT * FROM member WHERE ...` というSQL（データベースの ）に してくれます。SQLを 接書かなくていいのがORMの 。 はこの「データベースへの い合わせ」のこと。

- `if not member_data.authority:`
  - `member_data.authority` は、その人員の「管 権限フラグ」（True/Falseの ）。`not` を けているので「権限が いなら True」。
- 権限が ければ を します。「あなたが かは かったが、この操作は許可されていない」という意 の 号です。

> ⚠️ 401は「あなたが か からない（ログインして）」。403は「あなたが かは かったが、権限が足りない」。 アプリでは、 ログイン＝401、ログイン済みだが権限

> なし＝403、と使い　けています。

- 　　`except member.DoesNotExist:`
  - 28行　で人員が見つからなかったときに　んでくる先。`member.DoesNotExist` は「該　レコードなし」を　す専　のエラー。
- 　　人員が見つからなければ401を　します。

> 　　URLの　　に　いてくる「絞り込み条件」を取り出します。たとえば `?employee_no=123&shop=W1` のように　いてくる検索キーワードを　み取ります。

```
    #
    # URL  ?employee_no=                          None
    search_number = request.query_params.get('employee_no', None)
    # URL  ?shop=                              None
    search_shop = request.query_params.get('shop', None)
```

- 　　`request.query_params.get('employee_no', None)`
  - `query_params`（クエリパラメータ）は、URLの `?` 以降に　いてくる検索条件。たとえば `/api/member_list/?employee_no=123` なら、`employee_no` の　は `'123'`。
  - `.get('employee_no', None)` は「`employee_no` があれば取り出す、　ければ `None`」。　2引数の `None` が「見つからなかったときの　」です。

> 　　URLの `?` 以降に　　`=` `&` 　　`=` 　の形で　ける、検索・絞り込みのための　帯情　。`http://　/member_list/?shop=W1` の `shop=W1` の　　。「絞り込みの　　」だと思ってください。

- 　　に、所属（shop）での絞り込み条件を取り出します。

> 全人員データを取り出すのですが、そのとき
> ための工　をしています。一時　に「shop_order」
> という　べ　え　の隠し　号を各人に　け（異　・　社なら1、それ以外は0)、まずその　号
> 順、次に従業員　号順に　べます。

```
#
#
members = member.objects.annotate(
    # shop_order
    shop_order=Case(
        #     shop                    1
        When(shop='          ', then=Value(1)),
        #           0
        default=Value(0),
        #
        output_field=IntegerField(),
    )
# shop_order    →
).order_by('shop_order', 'employee_no')
```

ここは少し　しいので、　　に　解します。

- 　　`members = member.objects.annotate(`
  - `annotate`（アノテート＝　釈を　ける）は「各レコードに、計算した　を一時　な　として　加する」メソッド。ここでは `shop_order` という新しい　を各人員に　け足します。

> 「　釈・　　を　ける」。データベースから取ってくる各行
> に、その場限りの計算　を　り　けるイメージ。実際のテーブルには　は増えません。ここで
> は「　べ　え　の隠し　号」を一時　に　けています。

- 　　`Case(...)` の　身
  - `Case`（ケース）と `When`（ウェン）は「もし〜なら〜」という条件　岐をデータベース側で作る　。
  - 　　`When(shop='          ', then=Value(1))` …「もし所属が『異　・　社』なら、を `1` にする」。

- `default=Value(0)` …「上の条件に　てはまらなければ、　は `0`」。
- `output_field=IntegerField()` …「この `shop_order` という　は整数型ですよ」とDjangoに教えます。

- `.order_by('shop_order', 'employee_no')`
  - `order_by`（オーダーバイ）は「　べ　え」。
  - 1キー `shop_order` で　べると、`0`（現　）が先、`1`（異　・　社）が後ろになります。
  - じ `shop_order` の　では、　2キー `employee_no`（従業員　号）の小さい順に　びます。

> 純に従業員　号順に　べると、　社した人が現　の人に混ざってしまい見づらいからです。そこで「異　・　社」だけ一　の最後にまとめる。　近のコミット「人員一　で異　・　社を後ろに　更」がまさにこの処　です。

> ⚠️ `members` には「こういう　い合わせをするよ」という　が　っているだけで、実際のデータ取　はまだ起きていません。Djangoは「　に　になった瞬間」まで　い合わせを　らせます（　延価）。実際にデータが取られるのは、後のページネーション（§5.7）の　階です。

> §5.4で取り出した検索条件があれば、それに合うものだけに絞り込みます。従業員　号は「　一　」（含んでいればOK）、所属は「完全一　」で絞ります。

```
    #
    #
    if search_number:
      #
      members = members.filter(employee_no__icontains=search_number)
    #
    if search_shop:
      #
      members = members.filter(shop=search_shop)
```

- `if search_number:` … 検索　号が　　されていれば（空でなければ）True。
- `members.filter(employee_no__icontains=search_number)`
  - `filter`（フィルター）は「条件に合うものだけ残す」。
  - `employee_no__icontains` の `__icontains`（アイコンテインズ）は「　　」という条件。たとえば検索が `12` なら、`12`・`123`・`512` などすべてヒットします。

> 　　Djangoでは、フィールド　のうしろに `__` を　けて条件の種　を指　します。`__icontains`（含む）・`__gte`（以上）・`__lte`（以下）・`__exact`（完全一　）など。アンダースコア2個がポイントです。

- 　　所属の条件があれば、`shop=search_shop` で　　絞り込み。　号と違い、所属はピッタリ一　を求めます。

> 　　号は「12と　れたら12　　を　したい」という　　検索が　　。一　、所属は「W1」「W2」など決まった　なので、ピッタリ一　のほうが正確だからです。

> 　　絞り込んだ全データを一　に全　　すと重いので、「今のページ　」だけ切り出します（ページ送り）。切り出したデータをJSON形式にし、「全　で何件あるか」などの情　と一緒に　します。

```
    #
    #              1
    paginator = CustomPagination()
    #
    result_page = paginator.paginate_queryset(members, request)
    #                   JSON
    serializer = MemberSerializer(result_page, many=True)

    #              JSON
    return paginator.get_paginated_response(serializer.data)
```

- `paginator = CustomPagination()`
  - §8で詳しく見る `CustomPagination` を1つ作ります。この瞬間に「1ページ何件　示するか」が、管　者設　から　　に決まります。
- `result_page = paginator.paginate_queryset(members, request)`
  - `paginate_queryset`（ページネイト・クエリセット）は「全データ `members` から、`request` が　求するページ　号の　だけ切り出す」メソッド。
  - たとえば1ページ20件で2ページ　を　求していれば、21〜40件　を　します。
- `serializer = MemberSerializer(result_page, many=True)`
  - 切り出した人員データ（　数件）を `MemberSerializer` に　して、JSONに　　する準　をします。
  - `many=True`（メニー・トゥルー）は「対象は1件ではなく　　だよ」という指　。1件のときは　けません。
- `return paginator.get_paginated_response(serializer.data)`
  - `serializer.data` が、　　済みのJSONデータ（人員のリスト）。
  - `get_paginated_response` が、それに「全　で何件か（count)」「1ページ何件か（page_size)」などを足して、最終　な応　を組み　てます（　身は§8)。

> 　　React側がこのAPIを呼ぶと、次のようなJSONが　ってきます（※説　　の簡易　）。
>
> ```
> {
>   "count": 53,
>   "page_size": 20,
>   "results": [
>     {"employee_no": 101, "name": "　　", "shop": "W1", ...},
>     {"employee_no": 102, "name": "　　", "shop": "W1", ...}
>   ]
> }
> ```
>
> `count` が全件数、`results` が今のページ　。Reactはこれを　に　べ、ページ　号を作ります。

> 「新規　　」ページを開いたとき、　　フォームの初期　示　にログイン者自身のデータを　します。ここでも先に権限チェックを行い、権限が　ければ　きます。

```
#
#         API
class MemberNew(APIView):
  #                       GET
  def get(self, request):
    #
    login_no = request.session.get('login_No')
    #
    if not login_no:
      # 401
      return Response({'status': 'error', 'message': '
    '}, status=status.HTTP_401_UNAUTHORIZED)

    try:
      #
      #
      member_data = member.objects.get(employee_no=login_no)
      #                 Menu
      #
      if not member_data.authority:
        # 403
        return Response({'status': 'error', 'message': '                          '},
status=status.HTTP_403_FORBIDDEN)
    #
    except member.DoesNotExist:
      #
      # 401
      return Response({'status': 'error', 'message': '                          '},
status=status.HTTP_401_UNAUTHORIZED)

    #                   JSON
    serializer = MemberSerializer([member_data], many=True)
    #
    return Response(serializer.data)
```

- `class MemberNew(APIView):` … 新規　　を　　するクラス。

- … §5.3とほぼ じ「ログイン確 →権限確 」の チェック。 アプリの全ビューに共 する「 」です。
- `serializer = MemberSerializer([member_data], many=True)`
  - `[member_data]` と角カッコで囲って し、`many=True` を けています。Reactが「常にリストで受け取る」前 なので、1件でもリスト形式に えています。
- `return Response(serializer.data)`
  - ステータスコードを指 していません。この場合、DRFは自 で を けます。

ここがCRUDの「C（Create＝作る）」の 体です。 いので、ブロックに けて みます。

> Reactから送られてきた データを受け取り、まず「その従業員 号がすでに 済みでないか」を確 します。すでにあれば、エラー（400）を して を止めます。

```
#                          POST
def post(self, request):
  #
  data = request.data

  #
  if member.objects.filter(employee_no=data.get('employee_no')).exists():
    #       400
    return Response({'status': 'error', 'message': '
          '}, status=status.HTTP_400_BAD_REQUEST)
```

- `def post(self, request):`
  - メソッド が `post` なので、 （＝「データを送るので 存して」という書き込みのお願い）が たときに呼ばれます。

> 「データを送って新しく作る」ためのHTTPメソッド。 ポストに手紙を するイメージ。GETが「 み取り」なのに対し、POSTは「新規作成」。

- `data = request.data`
  - `request.data` には、Reactが送ってきた　フォームの　身（JSON）が、Pythonの辞書として　っています。
- `if member.objects.filter(employee_no=data.get('employee_no')).exists():`
  - `data.get('employee_no')` で、送られてきた従業員　号を取り出します。
  - `member.objects.filter(employee_no=...)` で「その　号の人員」を絞り込み、`.exists()` で「1件でも存在するか？」を True/False で確　します。
  - `.exists()` は実際のデータを取らず「あるかないか」だけ　べるので軽い、というのもポイント。
- すでに　じ　号があれば、　を　して　します。

> 「あなたの送ってきた　に　があります」を　す　号。　ミス・重　・形式違　などはこれ。サーバー側の故障（500　）ではなく、　を　します。

> 新しい人員には、所属に応じた「　準の休憩時間」をあらかじめセットします。所属が「現場系（W1, W2, A1, A2, J, 組　以上）」なら現場　の時間割を、それ以外（事　系など）なら　の時間割を、24個の休憩　に一括で書き込みます。これにより、　者が　回休憩を手　しなくて済みます。

```
    #
    if data.get('shop') in ['W1', 'W2', 'A1', 'A2', 'J', '        (W,A)']:
      # data
      data.update({
        #     1      11:40  12:40
        'break_time1': '#11401240',
        #     1                1
        'break_time1_over1': '#17201735',
        #     1                2
        'break_time1_over2': '#23350035',
        #     1                3
        'break_time1_over3': '#04350450',
        #     2
        'break_time2': '#14101510',
        # …         break_time2_over1    break_time6_over3                    …
        #     6                3
        'break_time6_over3': '#12201230',
      })
    #
    else:
      #
      data.update({
        #     1
        'break_time1': '#10401130',
        # …                   24            …
        #     6                3
        'break_time6_over3': '#12201230',
      })
```

⚠️ 上のコードは紙　の　合で　　示しています。実ファイル（91〜144行　）では、`if` 側・`else` 側ともに　　がすべて示　に書かれています。　　も　じ `'　　': '#　'` の形式が　んでいるだけで、構造は わりません。

- `if data.get('shop') in ['W1', 'W2', 'A1', 'A2', 'J', '        (W,A)']:`
  - `in [...]` は「その　がリストの　に含まれるか」を　　します。所属が現場系の6種のどれかなら True。
- `data.update({...})`
  - `update`（アップデート）は、辞書 `data` に新しいキーと　をまとめて　加・上書きするメソッド。ここでは24個の休憩　を一気にセットしています。

- … `'#11401240'` のような　字　は、　アプリ　自の時間　現です。`#` の後ろを4桁ずつ区切って「`1140` ＝11時40　」「`1240` ＝12時40　」と　み、「11:40〜12:40が休憩」を　します。`_over1` 〜 `_over3` は、残業の　　で休憩時間がずれるパターン違いです（詳しい意　は工数計算の章で扱います）。
- `else:`
  - 所属が現場系グループ　　の場合に実行される側。こちらも　　に24　を、　の　でセットします。

> 休憩時間は、現場と事　で時間割が違い、しかも残業の　で　わる　雑なものです。新規　　のたびに24個も手　　させるとミスが起きるので、所属を選べば　準　が自　で　るようにしている、という親切設計です。後で個　に　更することもできます。

> ⚠️　　よく見ると、`break_time4` 〜 `break_time6` 系の　は `if` 側と `else` 側で　じです。違うのは `break_time1` 〜 `break_time3` 系だけ。現場系と事　系で午前の休憩割が異なる、という業　ルールを　映しています。コードを　すときは、　　のブロックを　えて　す　　があるので　意してください。

> 完成したデータをシリアライザに　して「　身が正しいか」を検査します。検査に合格すればデータベースに　存し、201（作成成功）を　します。　合格なら400（　　エラー）を　します。

```
    #
    serializer = MemberSerializer(data=data)
    #
    if serializer.is_valid():
      #
      serializer.save()
      #               201
      return Response(serializer.data, status=status.HTTP_201_CREATED)

    #               400
    return Response({'status': 'error', 'message': '                    '},
status=status.HTTP_400_BAD_REQUEST)
```

- `serializer = MemberSerializer(data=data)`
  - `data=data` のように　　すのがポイント。これは「　　」の合図です。
  - （対して§5.7のように `MemberSerializer(result_page)` と　すと「　　」になります。　じシリアライザが、　し　で　割をえるのです。）
- `if serializer.is_valid():`
  - `is_valid`（イズ・バリッド＝妥　か）は「送られたデータがモデルの決まり（型・　須・桁数など）を　たしているか」を検査するメソッド。合格なら True。

> 「　　の検証・妥　性チェック」。たとえば「従業員　号は数字か」「　須　が　まっているか」を確　すること。　正なデータがデータベースに　るのを　ぐ「　　」の　　。`is_valid()` がこれを　います。なお151行　のメッセージは「バリ　ーション」と書かれていますが、これは　アプリ　の　記（タイプミス）で、　作には影響しません。

- `serializer.save()`
  - 検査に合格したデータを、　　。新規なので新しい行が1つ増えます。
  - ⚠️ この `save()` の瞬間に、§9で学ぶ　　し、「CREATE（新規作成）」の　　が `History` テーブルに残ります。ビューのコードには　　処　が一切書かれていないのに、自　で記　される――これがシグナルの　　です。

- `return Response(serializer.data, status=status.HTTP_201_CREATED)`
  - 存した を、 を けて します。

> 「新しいデータの作成に成功した」を す 号。 な る成功の200より具体 で、「新規作成が成功した」ことをはっきり示します。

- `is_valid()` が False（検査 合格）だった場合、ここまで て400を します。

CRUDの「U（Update＝更新）」を します。

> 「指 された従業員 号の人員を1件取ってくる」 共 処 を、小さな （メソッド）として切り出しています。見つからなければ `None` を します。更新と削除で何 も使うので、まとめておくと です。

```
#
#     API
class MemberUpdate(APIView):
  #       (pk)  1
  def get_object(self, pk):
    try:
      #             pk
      return member.objects.get(employee_no=pk)
    #
    except member.DoesNotExist:
      # None
      return None
```

- `def get_object(self, pk):`
  - `pk`（ピーケー）は の 。「1件を するための 」。ここでは従業員 号が の 割です。`pk` はURLから されます（§7参照）。

テーブルの　で、1行を一意に　　するための「　　号」。　じ　は2つと存在しません。図書館の　に　られたバーコードのようなもの。これがあれば「どの　（行）か」を間違いなく指　できます。

- `return member.objects.get(employee_no=pk)` … 従業員　号が `pk` の人員を取　してします。
- 見つからなければ `None`（なし）を　します。呼び出し側で「Noneなら　かった」と　　できます。

集画　を開いたとき、　集対象の人員データをします。ここでも「ログイン確　→権限確　→対象が存在するか確　」の順で　　チェックを行います。

```
  #                       GET  pk
  def get(self, request, pk):
    #
    #
    login_no = request.session.get('login_No')
    #
    if not login_no:
      # 401
      return Response({'status': 'error', 'message': '
    '}, status=status.HTTP_401_UNAUTHORIZED)

    #
    try:
      #
      member_data = member.objects.get(employee_no=login_no)
    #
    except member.DoesNotExist:
      # 403
      return Response({'status': 'error', 'message': '
  '}, status=status.HTTP_403_FORBIDDEN)
    #
    if not member_data.authority:
      # 403
      return Response({'status': 'error', 'message': '                          '},
status=status.HTTP_403_FORBIDDEN)

    #
    #
    member_instance = self.get_object(pk)
    #
    if member_instance is None:
      # 404
      return Response({'status': 'error', 'message': '                            '},
status=status.HTTP_404_NOT_FOUND)

    #
    # 1    JSON    many
    serializer = MemberSerializer(member_instance)
    #
    return Response(serializer.data)
```

- `def get(self, request, pk):`
  - §5.2の

- … おなじみの　　チェック。ログイン確　と権限確　です。
- `member_instance = self.get_object(pk)`
  - `self.get_object(pk)` で、§5.10.1の共　メソッドを呼び、　集対象を取　します。`self.` は「このクラス自身のメソッドを呼ぶ」という書き　。
- 対象が `None`（見つからない）なら　　を　します。

> 「　したけれど、そのデータが見つからない」を　す　号。Webで「ページが見つかりません」と出るあの404です。

- `serializer = MemberSerializer(member_instance)`
  - 1件だけなので `many=True` は　けません。1件のオブジェクトをそのままJSONに　　します。

> 集　　を受け取り、対象が存在するか確　し、検査に合格したら　存します。さらに「従業員　号を　の既存　号に　えようとしていないか」も　重チェックして、　号の重　を　ぎます。

```
  #                          PUT
  def put(self, request, pk):
    #
    #
    member_instance = self.get_object(pk)
    #
    if member_instance is None:
      # 404
      return Response({'status': 'error', 'message': '                          '},
status=status.HTTP_404_NOT_FOUND)

    #
    #
    data = request.data
    #
    serializer = MemberSerializer(member_instance, data=data)
    #
    if serializer.is_valid():
      #
      #
      if data.get('employee_no') != pk and
member.objects.filter(employee_no=data.get('employee_no')).exists():
        # 400
        return Response({'status': 'error', 'message': '
              '},status=status.HTTP_400_BAD_REQUEST)
      #
      serializer.save()
      # 200
      return Response(serializer.data, status=status.HTTP_200_OK)
    #               400
    return Response({'status': 'error', 'message': '                    '},
status=status.HTTP_400_BAD_REQUEST)
```

- `def put(self, request, pk):`
  - メソッド　が `put` なので、　　（＝「既存データを丸ごと差し　えて」という更新のお願い）が　たときに呼ばれます。

> 「既存のデータを更新（　き換え）する」ためのHTTPメソッド。POSTが「新規作成」なのに対し、PUTは「既にあるものを書き換える」。

- 更新対象を取　し、　ければ404。
- `serializer = MemberSerializer(member_instance, data=data)`

- ここが新規(POST)との違い。 を
  しています。
- これは「 」という更新モードの合図です。 1
  引数を さず `data=` だけなら新規、 1引数も せば更新、と覚えてください。

- `if serializer.is_valid():` … データの妥 性検査。合格なら次へ。
- `if data.get('employee_no') != pk and member.objects.filter(employee_no=data.get('employee_no')).exists():`
  - 2つの条件を `and`（かつ）で繋いでいます。
  - 左 `data.get('employee_no') != pk` … 「従業員 号を （元の `pk` と違う）」か。
  - 右 `member.objects.filter(...).exists()` … 「その新しい 号が
    」か。
  - が True、つまり「 号を えようとしていて、かつその 号が既に存在する」場合だけ、200行 で400を します。

> 号を えない更新（ 前だけ すなど）では、自 自身の 号は 然存在しているので、左の条件 `!= pk` で「 えていない」を先に して除外しています。これを れると「自 の 号と重 している」と誤 して、 の更新ができなくなってしまいます。

- `serializer.save()` … 更新を 存。この瞬間も signals が 火し、「UPDATE（更新）」の （ 更前後の差 つき）が残ります。
- 成功したら を します。新規の201と違い、更新は200を使います。

CRUDの「D（Delete＝消す）」を します。

> 削除対象の人員を取 し、存在すれば削除します。存在しなければ404を します。削除成功時は「 身のない成功（204）」を します。

```
#
#     API
class MemberDelete(APIView):
  # 1
  def get_object(self, pk):
    try:
      # pk
      return member.objects.get(employee_no=pk)
    #
    except member.DoesNotExist:
      # None
      return None


  #                            DELETE
  def delete(self, request, pk):
    #
    #
    member_instance = self.get_object(pk)
    #
    if member_instance is None:
      # 404
      return Response({'status': 'error', 'message': '                          '},
status=status.HTTP_404_NOT_FOUND)

    #
    #
    member_instance.delete()
    # 204
    return Response({'status': 'success', 'message': '                        '},
status=status.HTTP_204_NO_CONTENT)
```

- `def delete(self, request, pk):`
  - メソッド　が `delete` なので、　　（＝「このデータを消して」というお願い）が　たときに呼ばれます。

> 「指　したデータを削除する」ためのHTTPメソッド。　前のとおり「消す」。

- 　　削除対象を取　し、　ければ404。
- `member_instance.delete()`

- データベースから、その人員の行を　　します。この瞬間も signals が　火し、「DELETE（削除）」の　　が残ります（消す前に記　されるので、後から「　が何を消したか」を　えます）。

- `return Response({...}, status=status.HTTP_204_NO_CONTENT)`
  - 　　を　します。「成功したが、　す　身は　い」という意　。削除では「消したので、もう　すデータがない」ため204が　　です。

> 「処　は成功したが、　　（　すデータ）は　い」を　す　号。削除の成功時によく使われます。

> ⚠️　　4つのクラスを　して見ると、　アプリのビューには共　の型があります。　セッションで　　（ログイン・権限）→　対象データの取　→　シリアライザで　　・検査 →　　切なステータスコードで応　。この「型」さえ覚えれば、他の `kosu_views.py` や `team_views.py` も　じ　み　で　解できます。

> 　　シリアライザが「Pythonのオブジェクト」と「JSON」を双向に　　する係であること、そして　アプリの全シリアライザが驚くほど　く書けている　を　解します。

`serializers.py` の全　を見ます。

> 　　DRFのシリアライザ　　と、　アプリの全モデルを　み込みます。各シリアライザが「どのモデルを　　するか」を指　するために、モデルがなのです。

```
# DRF
from rest_framework import serializers
#                                    \
from ..models import member, Business_Time_graph, kosu_division, def_choice, \
                      #
                      team_member, inquiry_data, administrator_data, AsyncTask, History
```

- `from rest_framework import serializers` … DRFの `serializers`（シリアライザ）モジュールを　み込みます。
- 　　…　アプリの全モデル（member, Business_Time_graph …）を `kosu/models.py` から　み込みます。行　の `\`（バックスラッシュ）は「この行はまだ次の行に続く」という継続の印。　い import を見やすく折り　しているだけです。

> 　　memberモデルを、JSONに　　する係を　義します。たった4行ですが、これだけで「member の全項　をJSONにする・JSONをmemberにす・検査する」を全　こなします。

```

```

> をそのまま　　・検査ルールに使ってくれます。自　でいちいち書かなくて済む省　ツール。

- 　　`class Meta:`
  - `Meta`（メタ）は「このシリアライザの設　を書く　れ子のクラス」。　に「どのモデルを・どの項　を」　　するかを書きます。

> 「　体の　帯設　をまとめる小　屋」。シリアライザやモデルの　に`class Meta:`という　れ子を作り、対象モデルや対象項　などの設　を書きます。
> 　体の　作そのものではなく「設　メモ」だと思ってください。

- 　　`model = member` …　　対象は`member`モデルだと指　。
- 　　`fields = '__all__'` … `'__all__'`（オール）は「　　」という指　。従業員　号・　前・所属・24個の休憩　…すべてが自　で　　対象になります。一　だけにしたい場合は`fields = ['employee_no', 'name']`のようにリストで　挙します。

> 他のモデル（工数区　・工数データ・チーム・い合わせ・管　者設　・タスク・　　）についても、まったく　じ形で　　係を　義しています。　身はモデル　が違うだけです。

```
#             (kosu_division)
class DefSerializer(serializers.ModelSerializer):
  class Meta:
    model = kosu_division
    fields = '__all__'

#             (def_choice)
class DefChoiceSerializer(serializers.ModelSerializer):
  class Meta:
    model = def_choice
    fields = '__all__'

#           (Business_Time_graph)
class KosuSerializer(serializers.ModelSerializer):
  class Meta:
    model = Business_Time_graph
    fields = '__all__'

#       (team_member)
class TeamSerializer(serializers.ModelSerializer):
  class Meta:
    model = team_member
    fields = '__all__'

#           (inquiry_data)
class InquirSerializer(serializers.ModelSerializer):
  class Meta:
    model = inquiry_data
    fields = '__all__'

class AdministratorSerializer(serializers.ModelSerializer):#
(administrator_data)
  class Meta:
    model = administrator_data
    fields = '__all__'

#             (AsyncTask)
class TaskSerializer(serializers.ModelSerializer):
  class Meta:
    model = AsyncTask
    fields = '__all__'

#         (History)
class HistorySerializer(serializers.ModelSerializer):
  class Meta:
    model = History
    fields = '__all__'
```

- `DefSerializer` … 工数区　義 `kosu_division` の　係。
- `DefChoiceSerializer` … 工数区　の選　肢 `def_choice` の　係。
- `KosuSerializer` … 工数データ　体 `Business_Time_graph` の　係。　アプリの心データです。
- `TeamSerializer` … チーム　成 `team_member` の　係。
- `InquirSerializer` …　い合わせ `inquiry_data` の　係。
- `AdministratorSerializer` … 管　者設　`administrator_data` の　係。
- `TaskSerializer` …　期タスク `AsyncTask` の　係。
- `HistorySerializer` …　更　`History` の　係。`model = History`、`fields = '__all__'`。

> `ModelSerializer` + `fields = '__all__'` の組み合わせが強　だからです。モデルさえきちんと作ってあれば、　・検査ルールは全　自　。これがDjango/DRFの「楽さ」の正体です。　アプリは全モデルでこのパターンを　しています。

> ⚠️ 全項　を出すので楽ですが、「外　に見せたくない項　（パスワードなど）」まで出てしまう危険もあります。　アプリの member には機　項　が　いので　ありませんが、もし　の　を　加したら、`fields` をリスト指　に　えて隠す　が　です。

> URLとビューを結びつける「　線　」の　み　。`path` の3素、`<int:pk>`、`as_view()`、最後の `static()` の意　を　解します。

`urls.py` は「どのURLが　たら、どのビューに繋ぐか」を一　にしたファイルです。　の交換のようなものです。

URL　義に　　な path 関数と、振り　け先とな る各ビューモジュール、そして　　でメディアファイルを　信するための　　を　み込みま す。

```
# URL          path
from django.urls import path
#
from .views import main_views
#
from .views import kosu_views
#
from .views import member_views
#
from .views import team_views
#
from .views import def_views
#
from .views import inquiry_views
#
from kosu.views import asynchronous_views
#       MEDIA_URL
from django.conf import settings
#                static
from django.conf.urls.static import static
```

- from django.urls import path … URLを1　　義する path （パス）関数を　み込みま す。
- 　　… 振り　け先になる各ビューモジュールを　み込みます。機　ごとにファイルが　かれているので、それぞれ　み込みます。
- 　　… 後で（105行　で）メディアファイル　信に使う settings と static を　み込みます。

urlpatterns というリストに、URLとビューの対応を1行ずつ　べます。リストの上から順に照合され、最初に一　したものが採　されます。

```
# URL
urlpatterns = [
  # /login/ → Login
  path('login/', main_views.Login.as_view(), name='login'),
  # /logout/ → Logout
  path('logout/', main_views.Logout.as_view(), name='logout'),
  # …        …
  # /member_list/ → MemberList
  path('member_list/', member_views.MemberList.as_view(), name='member_list'),
  # /member_new/ → MemberNew
  path('member_new/', member_views.MemberNew.as_view(), name='member_new'),
  # /member_update/123/ → MemberUpdate
  path('member_update/<int:pk>/', member_views.MemberUpdate.as_view(),
name='member_update'),
  # /member_delete/123/ → MemberDelete
  path('member_delete/<int:pk>/', member_views.MemberDelete.as_view(),
name='member_delete'),
  # …                URL            …
]
```

`path()` には3つの　素を　します。　章の主　、人員系の4行を　に　み解きます。

- `'member_list/'`
  - ブラウザがこのパスに　たら、という「URLのパターン」。
- `member_views.MemberList.as_view()`
  - `MemberList` クラスを「URLに繋げる形」に　換しています。
  - `as_view()` （アズ・ビュー）が、その　換を行う決まり　句です（次項で詳説）。
- `name='member_list'`
  - このURLに　ける「あだ　」。プログラムの他の場所から「`member_list` という　前のURL」と呼び出せます。　接URL　字　を書くより、　前で参照するほうが、後でURLを　えても壊れにくくなります。

> 「クラスベースのビューを、URLに繋げられる関数の形に換する」決まりのメソッド。`MemberList` はクラスですが、URLは「関数」を期待していま

> す。そのギャップを　めるのが `as_view()`。クラスを関数に「　身」させる　　だと思ってください。

> 　　Djangoの　　では、URLに対して「呼び出せる関数」を　　する仕組みになっています。`MemberList` はクラスなので、そのままでは　　できません。`as_view()` を　けると「リクエストが　たら、　切な `get` / `post` / `put` / `delete` を呼び　ける関数」が作られ、それが　　されます。

> 　　`member_update/<int:pk>/` の `<int:pk>` は、URLの一　を「　数」として受け取る書き　です。`/member_update/123/` にアクセスすると、`123` が整数として `pk` という　前でビューに　されます。

- `<int:pk>` の　解：
  - `< >` …「ここは固　　字ではなく、　わる　　（　数）だよ」の合図。
  - `int` …「ここに　るのは　　だよ」という型の指　。`int` 以外に `str`（　字　）、`slug` などもあります。
  - `pk` … 受け取った　を　れる　　。ビューの `def put(self, request, pk):` の `pk` にそのまま　されます。

> ⚠️　　URLで `<int:pk>` と書いたら、ビューのメソッドも `def put(self, request, pk):` と、　じ `pk` という　前で受け取る　　があります。前が食い違うとエラーになります。§5.10で見た `MemberUpdate.put(self, request, pk)` の `pk` が、まさにこのURLから　される　です。

> 　　ブラウザが `PUT /api/member_update/123/` を送ると…
>
> 1. `urls.py` が `member_update/<int:pk>/` と照合し、`pk=123` を取り出す
> 2. `MemberUpdate.as_view()` が「PUTなので put メソッドを呼ぶ」と

3. `MemberUpdate.put(self, request, pk=123)` が実行される という れで、123 の人員が 更新されます。

バックアップ・ 元系のURLが に んでいま すが、 。 わりに `name=`（あ だ ）を1 ずつ えてあり、ビュー側はこの 前を見て「今どの処 か」を します（§11で詳説）。

```
  #
  path('kosu_backup/', asynchronous_views.backup, name='kosu_backup'),
  #               backup
  path('kosu_delet/', asynchronous_views.backup, name='kosu_delet'),
  #               backup
  path('kosu_load/', asynchronous_views.backup, name='kosu_load'),
  # …        def, choice, member, team, inquiry, setting, AsyncTask, History
backup/load          …
  #
  path('check_backup_status', asynchronous_views.check_task_status,
name='check_member_backup_status'),
  #
  path('download_backup', asynchronous_views.download_file,
name='download_member_backup'),
```

- … ここでは `asynchronous_views.backup` のように、`.as_view()` を けていません。これは `backup` が だからです（§11参照）。関数ビューはそのまま書けます。
- … 18 ほどのURLがすべて `backup` を指しますが、`name=` だけが異なります。ビュー側で `request` から「今のURLの 前」を み取り、処 を 岐します（§11.2）。

バックアップで作ったファイル（メディアファイル）を、ブラウザからダウンロードできるようにURLを 加します。

```
#                URL
urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

- `urlpatterns += ...` … 既存のURL一　に、メディア　信　のURLを　　ています。
- `static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)` … `MEDIA_URL`（`/media/`）に　たアクセスを、`MEDIA_ROOT`（実ファイルの　存フォルダ）に繋ぎます。これにより、生成したバックアップExcelをダウンロードできます。`MEDIA_URL` と `MEDIA_ROOT` の身は§12で確　します。

> 　　一　を「20件ずつ」などに　けるページ送りの仕組みと、アプリが「1ページの件数を管　者設　から　　に決めている」工　を　解します。

> 　　のデータを「1ページ◯件ずつ」に　けて　示すること。Google検索の結果が10件ずつページ　けされているのと　じ。全件を一　に　すと重いので、　　な　だけ切り出します。

`main_utils.py` の前　（CustomPagination）を　みます。

> 　　ページネーションの　　クラスや、応　を作る　、そして「1ページの件数」を　み取るための管　者設　モデルを　み込みます。

```
#
import inspect
#
from rest_framework.pagination import PageNumberPagination
#
from rest_framework.response import Response
#
from django.db import models
#         models                    myapp_models
from kosu import models as myapp_models
#                   1
from ..models import administrator_data
```

- `PageNumberPagination`（ページナンバーページネーション）… 「1ページ　・2ページ…」とページ　号で送る　式の　　クラス。これを継承します。
- `administrator_data` … 管　者設　モデル。ここに「1ページ何件　示するか（menu_row）」が　存されています。

> 　　ページ送りの設　を、固　　ではなく
> 　　決めるクラスです。作られた瞬間に管　者設　の最新レコードを見て「1ページ何件か」を取り込み、応　には「全件数・1ページ件数・データ　体」を含めます。

```
#
# PageNumberPagination
class CustomPagination(PageNumberPagination):
  #                                          #       1      20
  page_size = 20

  #
  def __init__(self):
    # administrator_data
    #
    last_record = administrator_data.objects.order_by("id").last()
    #
    if last_record is not None:
      #         1
      self.page_size = last_record.menu_row

  #
  def get_paginated_response(self, data):
    return Response({
      #                     #
      'count': self.page.paginator.count,
      #                                    # 1
      'page_size': self.page_size,
      #                                        #
      'results': data,
    })
```

- `class CustomPagination(PageNumberPagination):` … 準のページネーションを継承し、 自 を作ります。
- `page_size = 20` … 既 （デフォルト）は20件。後で上書きされなければこの が使われます。
- `def __init__(self):`
  - `__init__`（イニット）は「インスタンスが作られた瞬間に自 で呼ばれる初期化メソッド」。`paginator = CustomPagination()`（§5.7）の `()` の瞬間に、ここが走ります。

> クラスから実体（インスタンス）が作られる瞬間に1回だけ自 実行される、準 のためのメソッド。前後のアンダースコア2個（ダンダー）が 印。家を建てた瞬間に「 気と水 を す」初期工事のイメージ。

- `last_record = administrator_data.objects.order_by("id").last()`
  - 管　者設　を `id` 順に　べ、`.last()` で　　を取　します。これが現在効な設　です。
- `if last_record:` … 設　が存在すれば（`None` でなければ）、その `menu_row`（1ページ　示件数）で `self.page_size` を上書きします。

> 「1ページに何件　示するか」を、コードを書き換えずに　ようにするためです。管　者が「1ページ30件にしたい」と思ったら、設　を　えるだけ。次に作られる `CustomPagination` から自　で　映されます。

- `def get_paginated_response(self, data):`
  - §5.7の最後で呼ばれていたメソッド。　準では `next` / `previous`（次/前ページのURL）が含まれますが、　アプリは　自に上書きして、`count`・`page_size`・`results` の3つだけのシンプルな形にしています。
- `'count': self.page.paginator.count` … 絞り込み後の　　。Reactがページ　号の総数を計算するのに使います。
- `'page_size': self.page_size` … 1ページの件数も応　に含め、React側が　握できるようにしています。
- `'results': data` … 今のページのデータ　体（シリアライズ済みのJSON）。

> ```
> { "count": 53, "page_size": 20, "results": [ ... ] }
> ```
>
> この `count`・`page_size`・`results` は、まさにこの `get_paginated_response` が組み　てたものです。

> `main_utils.py` には他に `validate_employee_no_logic`（従業員　号の検証）や `get_all_model_names_in_myapp`（全モデル　取　）という関数もありますが、これらはバッ

> クアップ系やチーム系の　機　で使われるユーティリティです。　章の主　（人員ビューとページネーション）の　筋ではないので、ここでは深　りしません。

>

>

>

```
#
from threading import local
# 3
from django.db.models.signals import pre_save, post_save, post_delete
#
from django.dispatch import receiver
#               History
from .models import History, member, Business_Time_graph, team_member, kosu_division,
def_choice, administrator_data, inquiry_data
#
from .middleware.clear_session_middleware import get_current_request
#
from django.db import models

#
#
_thread_locals = local()
```

- … 3つのシグナルを 込 ます。
  - `pre_save` （ グ ブ セ の ブ ） ブ 後 の 存 の に ス 火 。 セ ブ ・ = = ・
  - `post_save` （ ポ ス ト 3 セ ・ ク セ ー ブ ブ ブ ダ を を 存 の 2 の 後 は 後 。 火 存 ブ 。 存 ブ を を を
  -

> 「 存する 前に、 更前の をデータベースから取って一時 管しておく」関数と、「その 管した を後で取り出す」関数のペアです。更新後に「どこがどう わったか（差 ）」を計算するために、 わる前の姿を覚えておく があるのです。

```
#
#
def set_instance_cache(instance):
  #
  model = type(instance)
  try:
    #               →
    # DB
    _thread_locals.instance_cache = model.objects.get(pk=instance.pk)
  #           DB
  except model.DoesNotExist:
    #                     None
    # None
    _thread_locals.instance_cache = None

#
#
def get_instance_cache():
  #                               None
  return getattr(_thread_locals, 'instance_cache', None)
```

- `def set_instance_cache(instance):`
  - `instance`（インスタンス）は「これから 存される、1件のデータ」。
- `model = type(instance)` … その `instance` が何のモデルか（member なのか kosu なのか）を べます。
- `_thread_locals.instance_cache = model.objects.get(pk=instance.pk)`
  - に、データベースに残っている を取 し、ロッカー（スレッドローカル）にしまいます。これが「 更前の姿」のスナップショットです。
- … 新規作成で、まだDBに古い が い場合は `None` をしまいます。

- `get_instance_cache()` … しまっておいた更新前の　を取り出します。`getattr(..., None)` で「　ければ None」を　す安全な取り出し　です。

ここが　　機　の　　です。「更新前」と「更新後」を全項　で　べ、　わったところだけを記にまとめます。少し　いので　階　に　みます。

> 差　を　れる空の　れ　を　意し、§9.2でしまった「更新前の　」を取り出します。さらに、取り出した古い　が「　　に今のデータと　じ種　・　じ1件のものか」を確　し、もし食い違っていたら誤　較を　けるため捨てます（None 扱い）。

```
#
#                     created=
def get_changes(instance, created):
  #
  changes = {}

  # 1.
  #
  old_instance = get_instance_cache()

  #                                                    PK
            None
  #       PK
  if old_instance and (not isinstance(old_instance, type(instance)) or old_instance.pk
!= instance.pk):
     #
     old_instance = None
```

- `def get_changes(instance, created):`
  - `created`（クリエイテッド）は「これは新規作成か？」を　す True/False。新規なら全項が「新しい　」、更新なら「　わった項　だけ」を記　する、という　岐に使います。
- `changes = {}` …　更　　をためる空の辞書。
- `old_instance = get_instance_cache()` … §9.2でしまった更新前の　を取り出します。

- … `isinstance(old_instance, type(instance))`（種　が　じか）と `old_instance.pk != instance.pk`（　じ1件か）を確　。どちらか食い違えば、　　を　べてしまう危険があるので `old_instance = None` にして捨てます。これは　数リクエストが混線したときの　険（コメントの「【修正】」が示すように、後から　強された安全策）です。

> 　　モデルの全項　を1つずつ回り、「新しい　」と「古い　」を取り出します。新規なら古い　はNone・全　　更扱い。更新なら、キャッシュがあればそれと　較、　ければ　のためDBから　接古い　を取り　して　較します。

```
#
#                 1
for field in instance._meta.fields:
  #
  field_name = field.name

  #
  #
  new_value = getattr(instance, field_name)

  #
  #
  if created:
    #
    old_value = None
    #
    is_changed = True
  #
  #
  else:
    #           (      )
    if old_instance:
      #
      old_value = getattr(old_instance, field_name)
      #
      is_changed = (old_value != new_value)
    #
    else:
      #         pre_save                                          DB

      try:
        # DB
        old_db_instance = type(instance).objects.get(pk=instance.pk)
        #
        old_value = getattr(old_db_instance, field_name)
        #
        is_changed = (old_value != new_value)
        #             Relation
        #
        old_instance = old_db_instance
      #
      except type(instance).DoesNotExist:
        #
        continue
```

- `for field in instance._meta.fields:`

- `instance._meta.fields` は「そのモデルの全項　（フィールド）の一　」。`for` で1項　ずつ順　に処　します。

- 　　`field_name = field.name` … 今見ている項　の　前（　：`name`, `shop`）。
- 　　`new_value = getattr(instance, field_name)` … `getattr` で、その項　の　　を　に取り出します。

> 　　`getattr(　　, '　　')` で「オブジェクトの、その前の項　の　」を取り出す関数。`instance.name` と書く　わりに、項　　が　数のときに使えます。全項　をループで回すこの場　で　活　します。

- 　　… 新規作成（`created` が True）なら、古い　は `None`、全項　を「　更あり（is_changed=True）」として扱います。
- 　　… 更新で、キャッシュがあれば、そこから古い　を取り、新旧を `!=`（違うか）で　較。違えば　更ありと　　します。
- 　　… キャッシュが　い　険ルート。DBから古い　を取り　して　較し、以降の処　でも使えるよう `old_instance` に　れ　します。それでも見つからなければ `continue`（この項　は　ばす）。

> 　　更があった項　について、その　を「JSONとして安全に　存できる形」に整えます。他テーブルへのつながり（リレーション）や　　などはそのままだとJSONにできないので、ID・　字　・isoformat　字　などに　換します。

```
    #     or
    #
    if is_changed:

      # JSON
      #
      if field.is_relation and field.many_to_one:
        #     ID
        #             ID
        old_json_safe_value = getattr(old_instance, field.attname) if old_instance else
None
        #               ID
        new_json_safe_value = getattr(instance, field.attname)

      #
      #
      elif not isinstance(new_value, (str, int, float, bool, type(None))):
        #                                                  ID
        #
        if isinstance(new_value, models.Model):
          #         id+
          old_json_safe_value = {'id': old_value.pk, 'str': str(old_value)} if
old_value else None
          #           id+
          new_json_safe_value = {'id': new_value.pk, 'str': str(new_value)}
        # Datetime
        #     /
        elif isinstance(new_value, (models.DateField, models.DateTimeField)):
          #
          old_json_safe_value = old_value.isoformat() if old_value else None
          #
          new_json_safe_value = new_value.isoformat()
        #
        #
        else:
          #
          old_json_safe_value = str(old_value) if old_value else None
          #
          new_json_safe_value = str(new_value)

      # JSON
      #
      else:
        #
        old_json_safe_value = old_value
        #
        new_json_safe_value = new_value
```

- `if is_changed:` … 更があった項 だけが、この の処 に ります。 わっていない項 は記 しません(駄を省く)。
- `if field.is_relation and field.many_to_one:`
  - その項 が「他テーブルへのつながり(多対一のリレーション)」なら、関 先のオブジェクトそのものは重くてJSONにできないので、 だけを記 します。

> テーブル 士のつながり。「多対一」は「多くの工数記 が、1人の人員に づく」ような関係。工数記 (多) → 人員(一)。詳しくは 8章で扱いますが、ここでは「他テーブルへの参照は、つながり先のIDだけ控える」と 解すれば十 です。

- `elif not isinstance(new_value, (str, int, float, bool, type(None))):`
  - が「 字 ・整数・小数・真偽・None」のような 純な型 (雑な)場合の処 。
  - … モデルそのものなら、`{'id': ..., 'str': ...}`(IDと 字 現)の形で記 。
  - … ・ 時なら、`.isoformat()` で「2026-06-02」のような 準 な 字に 換して記 。
  - … それ以外の 雑な は、とりあえず `str(...)` で 字 にして記 。
- `else:` … 字 や数 などの基 型は、 換 なのでそのまま使います。

> は `changes` という に で 存されます。JSONは「 字 ・数 ・真偽・リスト・辞書」しか れられません。 オブジェクトや、他テーブルへのつながりは、そのままだとJSONにできずエラーになるので、 字 やIDに「JSONが食べられる形」へ 換しているのです。

> 換した を、 更 をためる辞書に書き込みます。新規作成のときは「新しい 」だけ、更新のときは「古い と新しい のペア」を記 します。最後にこの辞書を します。

```
        # 6. changes
        #
        if created:
          #
          #
          changes[field_name] = new_json_safe_value
        #
        else:
          #
          #
          changes[field_name] = {'old': old_json_safe_value, 'new': new_json_safe_value}

    #
    return changes
```

- … 新規作成では、項　→ 新しい　、という形で記　（古い　は　いので）。
- … 更新では、項　→ `{'old':　, 'new':　}` という形で、　更前後をペアで記　します。後から「　前が『山　』→『　』に　わった」と一　で　かります。
- `return changes` … 完成した差　辞書を　します。

「山　さんの所属を W1 → A1 に　更」した場合、changes はこうなります。

```
{ "shop": {"old": "W1", "new": "A1"} }
```

わった `shop` だけが、　更前後つきで記　されます。これが `History` テーブルに残り、管画　で監査できます。

ここまで作った　を、実際に「　存の前後・削除の後」というタイミングに結びつけます。member モデルを　に、　存前のキャッシュ・存後の　記　・削除後の　記　の3つを　します。

```
#
#   member
@receiver(pre_save, sender=member)
#
def cache_old_member_instance(sender, instance, **kwargs):
  #
  set_instance_cache(instance)
```

- `@receiver(pre_save, sender=member)`
  - `@`（アットマーク）で始まるのが　　（後述）。「すぐ下の関数を、`member` の `pre_save`（　存　前）に結びつけよ」という意　。
  - これにより、member が　存される　　に、自　で `cache_old_member_instance` が呼ばれます。

> 　　関数の上に `@oo` と書いて、その関数に　加の割を「　り　ける」仕組み。ここでは「この関数をシグナルの受け手にする」という　割をっています。プレゼントにリボン（飾り）を　けるように、関数に機　を装飾するイメージ。

- `def cache_old_member_instance(sender, instance, **kwargs):`
  - シグナルから呼ばれる関数。`sender`（送り手＝どのモデルか）、`instance`（対象データ）、`**kwargs`（その他の　帯情　をまとめて受ける）を受け取ります。
- `set_instance_cache(instance)` … §9.2の関数を呼び、更新前の　をしまいます。これが「　存　前」に走るので、上書きされる前の姿を確　できます。

> 　　「キーワード引数を何個でもまとめて受け取る」書き　。シグナルは色々な　帯情　を　してきますが、使わないものは `**kwargs` でまとめて受け　せます。「その他もろもろ」の受け皿。

```
#                              (member)
#   member
@receiver(post_save, sender=member)
#                    created
def log_create_update_member_history(sender, instance, created, **kwargs):
  #
  request = get_current_request()
  #
  session_data = request.session.get('login_No') if request else None

  #
  # §9.3
  changes = get_changes(instance, created)

  #
  #         CREATE          UPDATE
  operation = 'CREATE' if created else 'UPDATE'

  #
  # History          1
  History.objects.create(
    #
    operation=operation,
    #
    table_name='member',
    #               ID
    record_id=instance.id,
    #
    login_No=session_data,
    #
    changes=changes,
  )
```

- `@receiver(post_save, sender=member)` … member の にこの関数を結びつけます。
- `def log_create_update_member_history(sender, instance, created, **kwargs):`
  - `created` を受け取っているのがポイント。新規作成なら True、更新なら False。これで操作 種 を します。
- `request = get_current_request()` … §10の関数で「今処 のリクエスト」を取 。ここから操作者を ります。
- `session_data = request.session.get('login_No') if request else None`
  - リクエストがあればセッションから操作者の従業員 号を取り、 ければ None。`A if else B` は「条件ならA、でなければB」という1行の書き （三項演算子）です。

- `changes = get_changes(instance, created)` … §9.3で差　を計算。
- `operation = 'CREATE' if created else 'UPDATE'` … 新規なら 'CREATE'、更新なら 'UPDATE' の　字　を作ります。
- `History.objects.create(...)`
  - `History` テーブルに新しい　　を1件作ります。操作種　・テーブル　・対象ID・操作者・更　　をまとめて記　。

```
#                  (member)
#   member
@receiver(post_delete, sender=member)
#
def log_delete_member_history(sender, instance, **kwargs):
  #
  request = get_current_request()
  #
  session_data = request.session.get('login_No') if request else None

  #
  #       1
  History.objects.create(
    #       DELETE
    operation='DELETE',
    #
    table_name='member',
    #                     ID
    record_id=instance.id,
    #
    login_No=session_data,
    #
    changes=None,
  )
```

- `@receiver(post_delete, sender=member)` … member の　　に結びつけます。
- 　　… 削除の　　を記　。操作は `'DELETE'` 固　、`changes=None`（差　は　いので）。削除前のID（`instance.id`）はまだ参照できるので、「　が何　を消したか」が残ります。

> 　　member とまったく　じパターンの「　存後・削除後の　　記　」を、他の主　モデルにも繰り　し　　しています。コードの形はモデル　・

> テーブル　が違うだけで、構造は　一です。

`signals.py` の残り（117〜399行　）は、次のモデルそれぞれに対して、§9.4とそっくりの `post_save`（新規・更新）と `post_delete`（削除）のペアを　　しています。

`Business_Time_graph`（工数データ）	あり（差　記　）	あり	117〜199
`team_member`（チーム）	あり	あり	203〜239
`kosu_division`（工数区　　義）	あり	あり	243〜279
`def_choice`（区　の選　肢）	あり	あり	283〜319
`administrator_data`（管　者設　）	あり	あり	323〜359
`inquiry_data`（　い合わせ）	あり	あり	363〜399

> ⚠️ なお、`pre_save`（　存前キャッシュ）が　　されているのは `member`（111行　）と `Business_Time_graph`（117行　）の2モデルだけです。他のモデルでは、§9.3.2で見た「キャッシュが　ければDBから取り　す」　険ルートが　いて差　を計算します。

> 　　モデルごとに `@receiver(..., sender=oo)` で
> 　　からです。共　化する書き　もありますが、　アプリは「　んで　かりやすさ」を　先し、各モデルに　示　に書いています。後　者が「このモデルの　　はここ」とすぐ見つけられる　　があります。

> 1. ビュー（member_views.py）で `serializer.save()` や `.delete()` を呼ぶ

2. その瞬間、Djangoが自　で `pre_save` → `post_save`（または `post_delete`）シグナルを　火
3. signals.py に　　された関数が呼ばれ、`get_changes` で差　を計算
4. `History.objects.create(...)` で　　を永久　存

のに、全データ操作が自　で監査記　される——これがシグナルの威　です。

　　リクエストとビューの「あいだ（middle）」に挟まる　間処　。すべてのリクエストが、ビューに　く前と、応　が　る前に、　ず　過する関所の　。たとえば「セキュリティチェック」「セッション　元」「CORSチェック」などが順　に　びます。空港の「　安検査→出国審査→　乗」のような一　の関所だと思ってください。

§1の　で見た「　MIDDLEWARE を順に　過」の　　です。　アプリ　自のミドルウェアが `clear_session_middleware.py` です。

　　リクエストが　るたびに、その「今のリクエスト」をスレッド専　の　管庫にしまい、処　が終わったら　　けます。これにより、§9のシグナルのような「リクエストを　接受け取れない場所」でも「今　が操作しているか」を　ることができます。

```
#
import threading

#
#
_request_local = threading.local()
#                            HTTP

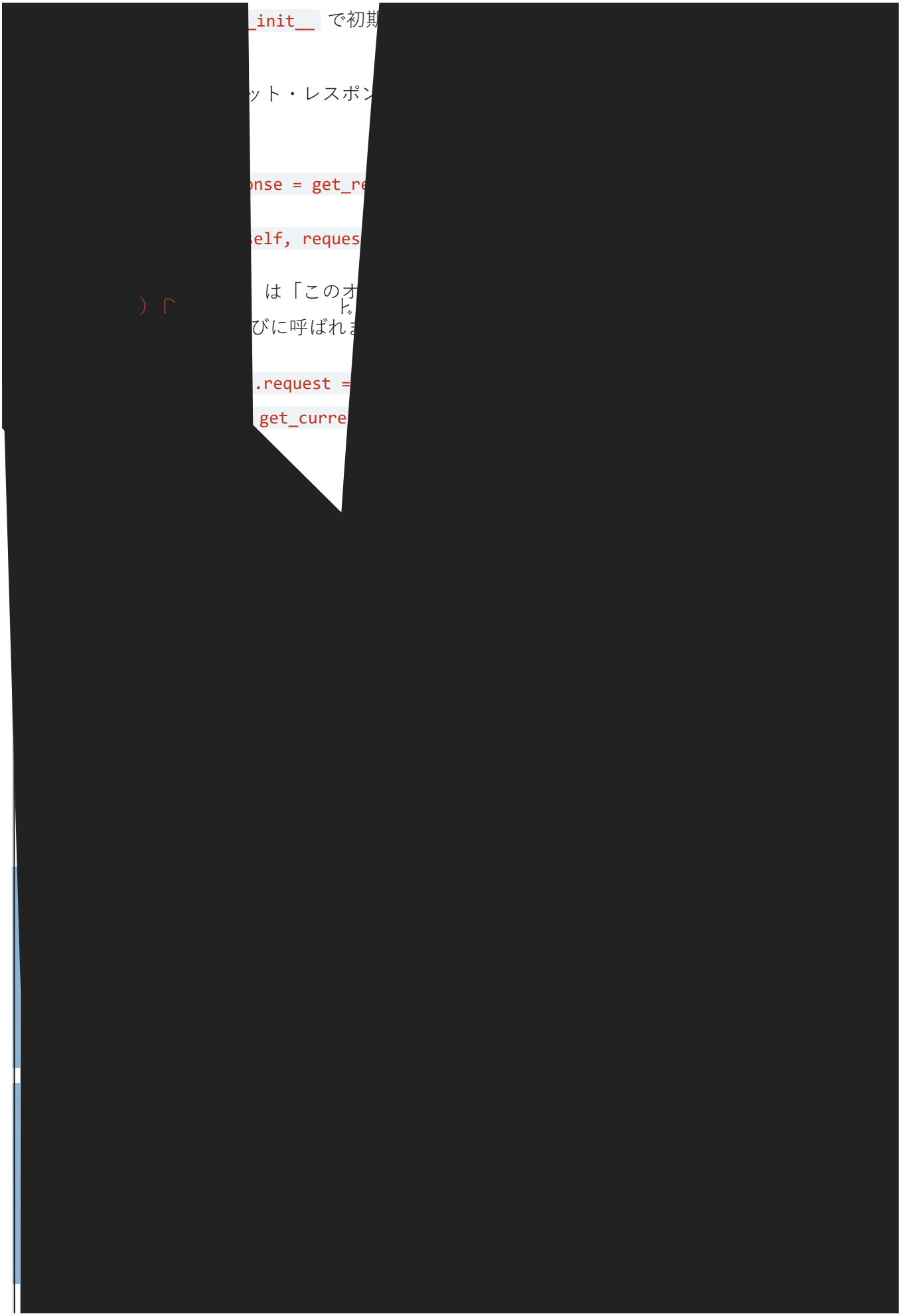
#
class CurrentRequestMiddleware:
  #
  #                 1
  def __init__(self, get_response):
    #                    get_respons
    #
    self.get_response = get_response

  #
  #
  def __call__(self, request):
    #
    #
    _request_local.request = request
    #                    get_respons
    #
    response = self.get_response(request)

    #                                                  (
          )
    #
    _request_local.request = None
    #
    return response

#                 HTTP
#
def get_current_request():
  #                                 'request'
  #         None
  return getattr(_request_local, 'request', None)
```

- `_request_local = threading.local()` … リクエストをしまうスレッド専　ロッカー（§9.1のスレッドローカルと　じ考え　）。
- `def __init__(self, get_response):`



このミドルウェアは、`settings.py` の `MIDDLEWARE` リストの　　に　　されています（§12で全体を見ます）。

```
    #
    'kosu.middleware.clear_session_middleware.CurrentRequestMiddleware',
```

リストの順　がそのまま「関所を　る順　」です。最後にあるので、セッションが　元された後にリクエストを　管できます。

> 　　アプリが起　して準　が整った瞬間に、`signals.py` を　み込み　ます。これをしないと、§9で書いたシグナル　　（@receiver）が実行されず、　　が一切残りません。

```
#
from django.apps import AppConfig

# kosu
class KosuConfig(AppConfig):
    #
    default_auto_field = 'django.db.models.BigAutoField'
    #            kosu
    name = 'kosu'

    #                      1
    def ready(self):
        # signals.py
        import kosu.signals
```

- 　　`class KosuConfig(AppConfig):` … kosu アプリ全体の設　クラス。
- 　　`default_auto_field = 'django.db.models.BigAutoField'` … 各テーブルの主キー（　号）に、　きな整数を自　で振る　式を既　にします。
- 　　`name = 'kosu'` … このアプリの　前。
- 　　`def ready(self):`

- `ready`（レディ＝準　完　）は「アプリの準　が整った瞬間に1回だけ呼ばれる」メソッド。
- `import kosu.signals`
  - ここで `signals.py` を　み込みます。

> ⚠️ `@receiver` デコレータは「そのファイルが　み込まれた瞬間」に　　処　を行います。`signals.py` はどこからも　　には import されないので、ここで意図　に　み込まないと、シグナルが一切　効になりません。「　　が残らない！」というトラブルの多くは、この `ready` での import　れが原因です。

> 「時間のかかる処　（数　件のバックアップなど）を、ユーザーを待たせずに　で走らせる」　　期処　のしくみ。タスクIDを　行し、　スレッドで実行し、ブラウザが後から状　を　い合わせる　れを　解します。

> 「時間のかかる処　を　で走らせ、終わるのを待たずに先に進む」やり　。たとえばレストランで「　　に30　かかる　」をしたとき、客を厨　の前に　たせ続ける（　期）のではなく、　号札を　して席で待ってもらい、できたら呼ぶ（　　期）　式。　アプリのバックアップ/　元はこの　式です。

> 期処　に　　な　準ツール（スレッド・一意IDの生成・一時ファイル　）、実際の処　　体（tasks.py の各関数）、タスク状　を　存するモデル、DRFの　　を　み込みます。

```
#
import os
#
import threading
#             ID
import uuid
#
import datetime
#     (sleep)
import time
#
import tempfile
#
from ..tasks import generate_kosu_backup, delete_kosu_data, load_kosu_file, \
                    #
                    generate_member_backup, load_member_file, generate_team_backup,
load_team_file, \
                    #
                    generate_def_backup, load_def_file, generate_choice_backup,
load_choice_file, \
                    #
                    generate_inquiry_backup, load_inquiry_file,
generate_setting_backup, load_setting_file, \
                    #
                    generate_AsyncTask_backup, delete_AsyncTask_data,
generate_History_backup ,delete_History_data
#
from ..models import AsyncTask
#
from rest_framework import status
#
from rest_framework.decorators import api_view, parser_classes
#
from rest_framework.parsers import MultiPartParser, JSONParser, FormParser
# URL
from django.urls import resolve
# JSON
from django.http import JsonResponse, FileResponse
```

- `import uuid` … `uuid`（ユーユーアイディー）は「世界で重　しない一意なID」を作る　具。各タスクに固　の　号札を　行するのに使います。
- `import tempfile` … アップロードされたファイルを一時　に　存する「一時ファイル」を作る　具。
- … `tasks.py` から、実際のバックアップ/　元/削除を行う関数群を　み込みます。これらが「重い処　の　体」です。

- `from ..models import AsyncTask` … タスクの状　（pending/success/error）を　存するモデル。　号札の控えを　管する　　です。
- `from django.urls import resolve` … URLパスから、そのURLの `name`（あだ　）を逆引きする　具。§7.5で「　じ関数・違う　前」だったのを思い出してください。

> 全バックアップ/　元/削除URLの共　　口です。まず一意なタスクIDを　行して　　に「処　　(pending)」で記　し、ブラウザに即座に　号札を　す準　をします。そして「今どのURL　で呼ばれたか」を逆引きして、実行すべき処　を　　します。

```
# POST
@api_view(['POST'])
#           JSON
@parser_classes([MultiPartParser, JSONParser, FormParser])
#
def backup(request):
  #       ID
  #             ID
  task_id = str(uuid.uuid4())
  #
  AsyncTask.objects.create(task_id=task_id, status='pending')

  #
  start_day = request.data.get('start_day')
  #
  end_day = request.data.get('end_day')

  # url_name
  #                 URL
  current_path = request.path
  #
  match = resolve(current_path)
  # URL  name
  url_name = match.url_name
```

- `@api_view(['POST'])`
  - `@api_view`（エーピーアイビュー）は「　　DRFのAPIビューにする」デコレータ。§5のクラスベースと違い、こちらは関数ビューです。`['POST']` で「POSTだけ受け　ける」と指

。

- `@parser_classes([MultiPartParser, JSONParser, FormParser])`
  - 受信データの形式を3種　解析できるようにします。`MultiPartParser`（マルチパート）は　　を扱える解析器。　元(load)ではExcelファイルが送られてくるので　です。

> ⚠️　`settings.py` の `REST_FRAMEWORK` 設　（§12）では、既　のパーサが `JSONParser` だけになっています。それだとファイルアップロードを受け取れません。そこでこの関数だけ `@parser_classes` でファイル対応の解析器を　加しているのです。

- `task_id = str(uuid.uuid4())` … `uuid.uuid4()` で一意なIDを作り、　字　にします。これがタスクの「　号札」。
- `AsyncTask.objects.create(task_id=task_id, status='pending')` …　　に「このタスクは処　　(pending)」と記　。
- 　　… 期間指　の処　　に、開始　・終　　を取り出します。
- 　　… `request.path`（今のURLパス）を `resolve` で逆引きし、`url_name`（あだ　）を取り出します。§7.5で見たとおり、URLは全　　じ `backup` 関数を指すので、　　します。

> §11.2で取り出したURL　（kosu_backup, member_load など）を `if/elif` で延々と　　し、それぞれに対応する「実行する関数」と「　す引数」を決めます。期間指　が　　なものは　　の検証も行い、ファイルアップロードが　　なものは一時ファイルに　存します。

　な3パターンだけを　粋して　みます（残りも　じ形の繰り　しです）。

```
    #
    if url_name == 'kosu_backup':
      #
      error_response = validate_dates(start_day, end_day)
      #
      if error_response:
        #
        return error_response
      #
      task_function = generate_kosu_backup
      #
      args = (start_day, end_day)
```

- `validate_dates(start_day, end_day)` … §11.5で見る 検証関数。 があればエラー応 が ります。
- … 検証でエラーが出たら、その場で して 。
- `task_function = generate_kosu_backup` … 実行すべき関数を 数に れておきます（まだ実行はしない）。
- `args = (start_day, end_day)` … その関数に す引数を 意。

```
  #
  elif url_name == 'kosu_load':
    #
    kosu_file = request.FILES.get('file')
    #
    temp_file_path = None
    try:
      #     xlsx
      with tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False) as temp_file:
        #                 (        )
        for chunk in kosu_file.chunks():
          #
          temp_file.write(chunk)
        #
        temp_file_path = temp_file.name
      #
      task_function = load_kosu_file
      #
      args = (temp_file_path,)
    #
    except Exception as e:
      #
      if temp_file_path and os.path.exists(temp_file_path):
        #
        os.remove(temp_file_path)
      # 500
      return JsonResponse({'status': 'error', 'message': f'                    :
{str(e)}'}, status=500)
```

- `request.FILES.get('file')` … `request.FILES` はアップロードされたファイルの　管場所。そこから `'file'` を取り出します。
- `with tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False)` … 一時　な `.xlsx` ファイルを作ります。`delete=False` は「withを　けても自　削除しない」（後で処　に使うため）。
- `for chunk in kosu_file.chunks():` …　きなファイルを `chunks()`（小　け）で少しずつ　み、書き込みます。一　に全　メモリに載せないための工　です。
- `temp_file_path = temp_file.name` … 作った一時ファイルの場所（パス）を覚えておきます。
- … 実行する関数とその引数（一時ファイルパス）を決めます。
- … もし書き込みでエラーが出たら、　の一時ファイルを消して（`os.remove`）、500エラーを　します。`f'...{str(e)}'` は　（　に　数を　め込める　字　）。

> 一時ファイルは「作業　の使い捨てファイル」。チャンクは「　きなデータを小　けにした塊」。　きなExcelを一気に　むとメモリを食うので、小　け（チャンク）で少しずつ処　します。引っ越しで　きな荷　を一　に運ばず、何回かに　けて運ぶイメージ。

```
  #
  elif url_name == 'member_backup':
    #
    task_function = generate_member_backup
    #
    args = ()
```

- 人員・チーム・区　などの「全件バックアップ」は期間指　が　　なので、`args = ()`（引数なし）でシンプルです。

```
  #     URL
  else:
    # 400
    return JsonResponse({'status': 'error', 'message': '                          '},
status=status.HTTP_400_BAD_REQUEST)
```

- どの `if/elif` にも該　しない場合は、　正なリクエストとして400を　します。

> §11.3で決めた「実行する関数」を、
> に　して走らせます。　体はその完　を待たず、すぐにブラウザへ「　号札（タスクID）」を　します。これでユーザーは待たされません。

```
    #
    thread = threading.Thread(target=handle_task, args=(task_id, task_function, *args))
    #
    thread.start()

    #       ID
    #               ID
    return JsonResponse({'status': 'success', 'task_id': task_id})
```

- `thread = threading.Thread(target=handle_task, args=(task_id, task_function, *args))`
  - `threading.Thread` で「 の作業員（スレッド）」を1人 意します。`target=handle_task` がその作業員にやらせる仕事（§11.7）。`*args` は§11.3で決めた引数を 開して します。
- `thread.start()` … 作業員に「始めて」と合図。
  体はその完 を待ちません。
- `return JsonResponse({'status': 'success', 'task_id': task_id})`
  - ブラウザには即座に「受け けました。 号札はこれです（task_id）」と します。ブラウザはこの 号札を使って、後から状 を い合わせます（§11.6）。

> 1. ブラウザ「バックアップして」→ サーバー「 号札123 ね（即 ）」＋ で処 開始
> 2. ブラウザ「123 どう？」→ サーバー「まだ処 （pending）」
> 3. （しばらく後）ブラウザ「123 どう？」→ サーバー「完成（success）！ファイルはここ」
> 4. ブラウザ「ダウンロードする」→ サーバーがファイルを す
>
> ユーザーは固まらず、待っている間も他の操作ができます。

> 期間指 の処 で、開始 ・終 が正しいかを検査します。空でないか・ の形式が正しいか・終 が昨 以前か・開始 が終 をえていないかを順にチェックし、 があればエラー応 を、 なければ None を します。

```
#
#
def validate_dates(start_day, end_day):
  #
  today_str = datetime.date.today().strftime('%Y-%m-%d')
  #
  if not start_day or not end_day:
    # 400
    return JsonResponse({'status': 'error', 'message': '                          '},
status=status.HTTP_400_BAD_REQUEST)

  try:
    #
    end_date_obj = datetime.date.fromisoformat(end_day)
    #
    today_date_obj = datetime.date.fromisoformat(today_str)
    #
    start_date_obj = datetime.date.fromisoformat(start_day)
  #
  except ValueError:
    # 400
    return JsonResponse({'status': 'error', 'message': '                        '},
status=status.HTTP_400_BAD_REQUEST)
  #
  if end_date_obj >= today_date_obj:
    # 400
    return JsonResponse({'status': 'error', 'message': '
    '}, status=status.HTTP_400_BAD_REQUEST)
  #
  if start_date_obj > end_date_obj:
    # 400
    return JsonResponse({'status': 'error', 'message': '
    '}, status=status.HTTP_400_BAD_REQUEST)
  #             None
  return None
```

- `today_str = datetime.date.today().strftime('%Y-%m-%d')` … 今　の　　を `2026-06-02` 形式の　字　で取　。
- 　　… 開始　・終　　のどちらかが空なら400。
- 　　… `fromisoformat` で　字　を　　型に　換。　換できなければ（形式　正）`except ValueError` で400。
- 　　… 終　　が　　　　なら「昨　まで」のルール違　として400。　　のデータはまだ確　していないので、バックアップ対象外という業　ルールです。
- 　　… 開始　が終　　より後（逆　）なら400。

- `return None` … すべて合格なら `None` を　します。§11.3では「`error_response` が None でなければエラー」と　　していたので、None＝合格の合図です。

> 　　ブラウザが　号札（タスクID）を持って「私の処　どうなった？」と　いてくる窓口です。　　を見て、成功なら結果ファイルの場所を、エラーなら　　を、まだなら「処　　(202)」を　します。

```
# GET
@api_view(['GET'])
#
def check_task_status(request):
  #                       ID
  task_id = request.GET.get('task_id')

  #       ID
  #
  if not task_id:
    # 400
    return JsonResponse({'status': 'error', 'message': '      ID
    '}, status=status.HTTP_400_BAD_REQUEST)

  try:
    #                       ID
    #
    task = AsyncTask.objects.get(task_id=task_id)
    #
    if task.status == 'success':
      #
      return JsonResponse({'status': 'success', 'file_path': task.result})
    #
    elif task.status == 'error':
      #
      return JsonResponse({'status': 'error', 'message': task.result})
    #
    else:
      #         (202)
      return JsonResponse({'status': 'pending'}, status=202)

  #     ID
  except AsyncTask.DoesNotExist:
    # 400
    return JsonResponse({'status': 'error', 'message': '            ID      '},
status=status.HTTP_400_BAD_REQUEST)
```

- `task_id = request.GET.get('task_id')` …　い合わせ対象のタスクIDを受け取ります。
- `task = AsyncTask.objects.get(task_id=task_id)` …　からそのタスクを取　。
- … 成功（success）なら、`task.result`（結果ファイルの場所）を　します。
- … エラーなら、`task.result`（エラー　）を　します。
- … まだ処　なら、　を　します。ブラウザはこれを見て「まだか、また後で　こう」と　します。

「リクエストは受　したが、処　はまだ完　していない」を　す　号。　　期処　で「処　　」を　えるのに使います。

§11.4で　スレッドに　された「実際の作業」です。指　された関数を実行し、結果を見て、　　のステータスを成功(success)かエラー(error)に更新します。　　で　期せぬエラーが起きても、　　にエラーとして記　します。

```python
#                  (      )
#
def handle_task(task_id, task_function, *args, **kwargs):
  try:
    #
    #
    result = task_function(*args, **kwargs)
    #
    is_explicit_error = (
      #
      isinstance(result, tuple) and
      #
      len(result) > 0 and
      #
      isinstance(result[0], dict) and
      # status  error
      result[0].get('status') == 'error'
    )

    #
    task = AsyncTask.objects.get(task_id=task_id)

    #
    if is_explicit_error:
      #
      #
      error_dict = result[0]
      #             error
      task.status = 'error'
      #
      task.result = error_dict.get('message', '
    ')
    #
    else:
      #
      #             success
      task.status = 'success'
      #
      task.result = result

    #
    #
    task.save()

  #
  except Exception as e:
    #
    try:
```

```
        #
        task = AsyncTask.objects.get(task_id=task_id)
        # error
        task.status = 'error'
        #
        task.result = str(e)
        #
        task.save()
    #
    except AsyncTask.DoesNotExist:
        #
        print(f"Error: AsyncTask with id {task_id} not found."
```

- `result = task_function(*args, **kwargs)` … §11.3で決めた実処　（バックアップ生成など）を実行します。ここが時間のかかる　。　　ので、ユーザーは待ちません。
- 　　… 結果が「タプルで・先　が辞書で・statusがerror」という形なら「　示　なエラー」と　。これは tasks.py 側の関数が「業　　な失　」を　すときの　束の形です。
- `task = AsyncTask.objects.get(task_id=task_id)` …　　から該　タスクを取　。
- 　　…　示　エラーなら、ステータスを `'error'`、メッセージを記　。
- 　　… 正常終　なら、ステータスを `'success'`、結果（ファイルパスなど）を記　。
- `task.save()` …　　を更新。これにより、§11.6の状　確　で正しい状　が　るようになります。
- 　　…　期せぬエラー（プログラムの　外）が起きた場合は、　　に `'error'` とエラー　字　を記　。　　すら見つからなければ、最後の手　としてログに出すだけにとどめます。

> 　　完成したバックアップファイルを、ブラウザにダウンロードさせる窓口です。ファイルを　し終わった後（接続が　じた後）に、3　待ってから一時ファイルを自　削除する後始　も仕込んでいます。

```
# GET
@api_view(['GET'])
#
def download_file(request):
  #
  file_path = request.GET.get('file_path')
  #
  file_handle = open(file_path, 'rb')
  #
  response = FileResponse(file_handle, as_attachment=True,
filename=os.path.basename(file_path))

  #
  def delayed_file_cleanup():
      # 3
      time.sleep(3)
      #
      if os.path.exists(file_path):
        try:
          #
          os.remove(file_path)
        #
        except Exception as e:
          #
          print(f"Cleanup failed after delay for {file_path}: {e}")

  #
  def cleanup_on_close():
      #
      thread = threading.Thread(target=delayed_file_cleanup)
      #
      thread.start()

  #
  response.close = cleanup_on_close

  #
  return response
```

- `file_path = request.GET.get('file_path')` … §11.6で受け取ったファイルの場所。
- `file_handle = open(file_path, 'rb')` … ファイルを `'rb'`（ み取り・バイナリ）で開きます。Excelなどは 字でなくバイナリなので `b` を けます。
- `response = FileResponse(...)`
  - `FileResponse`（ファイルレスポンス）で、ファイルをダウンロードさせる応 を作ります。`as_attachment=True` は「ブラウザで開かず、ダウンロード 存させる」指 。

`filename=os.path.basename(file_path)` で　存時のファイル　を指　します。

- `delayed_file_cleanup` … 3　待ってから一時ファイルを削除する　　関数。ダウンロードが終わる前に消すと壊れるので、少し待ちます。
- `cleanup_on_close` … 接続が　じたとき、削除を　スレッドで走らせる関数。
- `response.close = cleanup_on_close` … 応　の「　じる処　」を、後始　つきの関数に差し　えます。これで「ダウンロード完　→自　でファイル削除」が実現します。

> バックアップファイルは一時　なもので、ためておくとサーバーの　　を圧　します。ダウンロードが済んだら　　なので、自　で掃除しているのです。「使い終わった皿をすぐ下げる」イメージ。

> プロジェクト全体の「設　の集　　」。　章に関わる重　な設　（INSTALLED_APPS, MIDDLEWARE, CORS, REST_FRAMEWORK, SESSION）を　心に　みます。

`settings.py` は「アプリ全体の設　が全　書いてある場所」です。多いので、　章に関係するを　心に　みます。

> プロジェクトの　　となる場所・　　・デバッグモード・接続を許可するホスト　を設　します。

```
#
BASE_DIR = Path(__file__).resolve().parent.parent

#
env = environ.Env()
# .env
env.read_env(os.path.join(BASE_DIR, '.env'))

#                          .env
SECRET_KEY=env('SECRET_KEY')

#                     /      .env
DEBUG=env.bool('DEBUG')

#             UTF-8
DEFAULT_CHARSET = 'utf-8'

#
ALLOWED_HOSTS = [
    #     (Azure)
    'hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
    #
    'localhost',
    #       (         )
    '127.0.0.1'
    ]
```

- `BASE_DIR = Path(__file__).resolve().parent.parent` … プロジェクトの根っこの場所を計算。`.env` やデータベースの場所の基準になります。
- … `.env` ファイル（ の設 を書くファイル）を 込 ます。 やDBパスワードはコードに 書きせず、ここから ます。
- `SECRET_KEY=env('SECRET_KEY')` … 暗号化・署 に使う 。 れると危険なので `.env` から ます。

> SECRET_KEY は、セッションの署 などに使う「アプリの実印」。`.env` は 情 （ ・パスワード）を書く専 ファイルで、Gitに しない 束。実印を玄関に り出さないのと じで、コードに 接書きません。

- `DEBUG=env.bool('DEBUG')` … デバッグモード。開 は True（詳しいエラー画 が出る）、 は False（隠す）。

- `ALLOWED_HOSTS` … このアプリに接続してよいホスト　のリスト。許可外からのアクセスは拒　します。

> このプロジェクトで使う「機　パック（アプリ）」の一　です。Django　準の機　に加え、　アプリ　自の `kosu`、API　の `rest_framework`、　期　の `django_q`、CORS　の `corsheaders` などを　効にします。

```
#
INSTALLED_APPS = [
    #
    'django.contrib.admin',
    #
    'django.contrib.auth',
    #
    'django.contrib.contenttypes',
    #
    'django.contrib.sessions',
    #
    'django.contrib.messages',
    #
    'django.contrib.staticfiles',
    # Bootstrap4            UI
    'bootstrap4',
    #
    'bootstrap_datepicker_plus',
    #
    'kosu',
    #
    'django_q',
    # DRF  API
    'rest_framework',
    # CORS
    'corsheaders',
]
```

- `'kosu',` …　アプリ。これを　　することで、§10.4の `apps.py`（`KosuConfig`）が み込まれ、`ready` でシグナルが　効になります。
- `'django_q',` … Django-Q。　　期タスクの仕組み（　アプリではバックアップ系で使）。

- `'rest_framework',` … DRF。§5・§6のAPI機　の　　。
- `'corsheaders',` … CORS（後述）を扱う　　。

> §10で学んだ「関所の　」をここで　義します。リクエストは上から順に、応　は下から逆順に、各関所を　ります。　アプリ　自の CurrentRequestMiddleware は最後に　かれています。

```
#
MIDDLEWARE = [
    # ①CORS
    'corsheaders.middleware.CorsMiddleware',
    # ②
    'django.middleware.security.SecurityMiddleware',
    # ③
    'django.contrib.sessions.middleware.SessionMiddleware',
    # ④
    'django.middleware.common.CommonMiddleware',
    # ⑤CSRF
    'django.middleware.csrf.CsrfViewMiddleware',
    # ⑥
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    # ⑦
    'django.contrib.messages.middleware.MessageMiddleware',
    # ⑧
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    # ⑨
    'whitenoise.middleware.WhiteNoiseMiddleware',
    # ⑩    :
    'kosu.middleware.clear_session_middleware.CurrentRequestMiddleware',
]
```

- `'corsheaders.middleware.CorsMiddleware'` … CORSチェックを　　に　きます（フロントからのアクセス許可を真っ先に　　するため）。
- `SessionMiddleware` … セッションを　元する関所。これより後の処　で `request.session` が使えるようになります。§5で `request.session.get('login_No')` を使えたのはこの関所のおかげです。
- `CsrfViewMiddleware` … CSRF（後述）対策の関所。

- `CurrentRequestMiddleware` … §10.2の　自ミドルウェア。　　に　くことで、セッション　元後にリクエストを　管できます。

> ⚠️　　上から順に　るので、順　が　事です。たとえば `SessionMiddleware` より前で `request.session` を使おうとすると、まだ　元されていなくてエラーになります。むやみに　べ　えてはいけません。

> フロント（localhost:3000）とバック（localhost:8000）が　のアドレスでも、ブラウザの安全装　に　かれず　信できるようにする設　です。

```
#
CORS_ORIGIN_ALLOW_ALL = True
#         (          )
CORS_ALLOW_CREDENTIALS = True
#
CORS_ALLOW_ORIGINS = [
    # React
    'http://localhost:3000',
    # Django
    'http://localhost:8000',
    # Django(         )
    'http://127.0.0.1:8000',
]
```

> Cross-Origin Resource Sharing（クロスオリジン・リソース・シェアリング）。ブラウザの安全機　で、「　のアドレス（オリジン）への　信」を原則禁止しています。　アプリは開　時、フロントが `localhost:3000`、バックが `localhost:8000` とアドレスなので、このままでは　信が　かれます。CORS設　で「この　信元は許可」と　示して、　信できるようにします。「　の建　への　ち　り許可証」のイメージ。

- `CORS_ORIGIN_ALLOW_ALL = True` … すべての　信元からのアクセスを許可。開　時の　性のため。

- `CORS_ALLOW_CREDENTIALS = True` … 。これが いと、 アドレス間でログイン状 が てません。 アプリのセッション 証には 須です。
- `CORS_ALLOW_ORIGINS` … 許可する 信元の 示リスト。

> DRF全体の既 作を設 します。ページ送りの 式・1ページ件数・受信形式・応 形式を決めます。

```
# DRF
REST_FRAMEWORK = {
    #
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    #       1
    'PAGE_SIZE': 20,
    #
    'DEFAULT_PARSER_CLASSES': (
        # JSON
        'rest_framework.parsers.JSONParser',
    ),
    #
    'DEFAULT_RENDERER_CLASSES': (
        # JSON
        'rest_framework.renderers.JSONRenderer',
    ),
}
```

- `'DEFAULT_PAGINATION_CLASS'` … 既 のページ送り 式。ただし§5.7で見たように、人員ビューは 自の `CustomPagination` を使うので、この既 は上書きされます。
- `'PAGE_SIZE': 20` … 既 の1ページ件数。
- `'DEFAULT_PARSER_CLASSES'` … 既 では 。だから§11.2でファイルアップロードを扱う `backup` 関数は、わざわざ `@parser_classes` でファイル対応を 加していたのです。
- `'DEFAULT_RENDERER_CLASSES'` … 応 は 。HTMLの管 画 UIは出さず、純粋なJSONだけを すAPIにしています。

ログイン状　を覚えておくセッションの　管先や効期限、クッキーの安全設　を決めます。

```
#
SESSION_ENGINE = 'django.contrib.sessions.backends.db'
#                            10
SESSION_COOKIE_AGE = 315360000
#
SESSION_COOKIE_SAMESITE = 'Lax'
# HTTP
SESSION_COOKIE_SECURE = False
```

- `SESSION_ENGINE = 'django.contrib.sessions.backends.db'` … セッションを　に　存。§5で `request.session.get(...)` が　んでいたのは、このDB　存のセッションです。
- `SESSION_COOKIE_AGE = 315360000` …　効期限。315360000　≒ 10　。　くして「めったにログアウトされない」運　にしています。
- `SESSION_COOKIE_SAMESITE = 'Lax'` … クッキーを送る条件。`'Lax'`（緩め）は基な安全性を　ちつつ、　常の遷移では送る設　。
- `SESSION_COOKIE_SECURE = False` … HTTP（暗号化なし）でもクッキーを送る。開の設　で、　でHTTPSを使うなら True にすべき項　です。

⚠️　実ファイルの76〜77行　には `#SESSION_COOKIE_SAMESITE = 'None'` などコメントアウトされた行があります。これは　（HTTPS・クロスサイト）　の　設　で、開　は　効にしてあります。環境を切り　えるときに使う「　の設　」です。

最後に、　章で何　も出てきた「セッション　証」と「CSRF」を整　します。

アプリは　　を使っています。　れは次のとおり。

```
①
        ↓
②                  OK →              login_No=123          DB
        ↓
③                                ID
        ↓
④
        ↓
⑤                                 → request.session.get('login_No')   123
        ↓
⑥         §5
```

§5で何　も `request.session.get('login_No')` を呼んで「　がアクセスしているか」を確　していました。これが成り　つのは、ログイン時にセッションへ　号を　存し、以降のリクエストでクッキー（　）から　元しているからです。クッキーは `SessionMiddleware`（§12.3）が処　します。

ブラウザがサーバーから受け取り、次回以降のリクエストに自　で　　する「小さなメモ」。　アプリではセッションID（　）が　っています。手荷　かり所の「引換券」のイメージ。引換券を見せれば、　けた荷　（セッション）を出してもらえます。

Cross-Site Request Forgery（クロスサイト・リクエスト・フォージェリ＝サイト間リクエスト偽造）。悪意のあるサイトが、ログイン　のあなたのクッキ

> ーを悪　して、勝手にあなたの　前で操作を送りつける攻撃。たとえば「気づかぬうちに、あなたの権限で人員データが削除される」といった　害。

CSRF対策として、Djangoは「正規の画　からの送信であることを示す合言　（CSRFトークン）」を　求します。これを `CsrfViewMiddleware`（§12.3）がチェックします。`CSRF_TRUSTED_ORIGINS`（§12.7）に　　した　信元からの送信を信　します。

> セッション　証はクッキーで　人確　するので　　ですが、「クッキーは自　　される」という性質が悪　されると、　人になりすました操作が　ってしまいます。CSRFトークンという「正規画　でしか手に　らない合言　」を　　することで、偽の送信を　きます。「　（クッキー）だけでなく、その場で　行される合言　（トークン）も　求する」　重チェックです。

`401 Unauthorized` が　る	ログイン、またはセッション切れ（`login_No` がセッションに　い）	再ログインする。クッキーが送られているか確（§13.1）
`403 Forbidden` が　る	ログイン済みだが `authority`（権限）が False	管　者に権限　　を依　。member の authority を確
`404 Not Found` が　る	指　した `pk`（従業員　号）の人員が存在しない	URLの　号が正しいか、データが消えていないか確
`400 Bad Request`（　号重　）	/更新しようとした従業員　号が既存（§5.9.1, §5.10.3）	の　号にする
`400`（バリテーションエラー）	送信データがモデルの決まりを　たさない（`is_valid()` がFalse）	須　・型・桁数を確　。React側の送信　　を検
（History）が残らない	`apps.py` の `ready` で `import kosu.signals` が	`ready` のimportを確　。signals.py が　み込まれているか

	けている（§10.4）	
の差 が （前後が混ざる）	スレッド混線、またはキャッシュ れ	§9.3.1の安全確 ・§9.3.2の 険ルートで概ね止済み。再現するならロギングで う
CORSエラー（ブラウザのコンソール）	オリジン 信が許可されていない（§12.4）	`CORS_ALLOW_ORIGINS`

§7.4の `<int:pk>` について、`/member_update/123/` にアクセスしたとき、`123` はどのように、どの　前でビューに　されるか。ビュー側の受け取り　も含めて説　せよ。

§9で、ビューのコード（member_views.py）には　　を残す処　が1行も　いのに、　存・削除のたびに　　が `History` テーブルに残る。これを実現している仕組み（シグナルと apps.py の ready）を、　れに沿って説　せよ。

§10.2の `CurrentRequestMiddleware` は、なぜ「リクエストをスレッドローカルに　管」するがあるのか。これが　いと、§9のシグナルで何が困るか説　せよ。

§11で、`backup` 関数（§11.2）は処　の完　を待たずにすぐ `task_id` を　している（§11.4）。なぜ完　を待たないのか。待つ設計だと何が困るか説　せよ。

§12.3で、`SessionMiddleware` は `CurrentRequestMiddleware` より前に　んでいる。この順が逆だったら何が起きるか、§13.1のセッション　元の　れを　まえて考えよ。

§13で、セッション　証だけでなく CSRF 対策も　　なのはなぜか。「クッキーが自　　される」という性質と結びつけて説　せよ。

- 。　アプリは Django + DRF で、画　ではなく　として作られている。
- … ブラウザ → MIDDLEWARE（関所） → urls.py（振り　け） → ビュー（処　） → モデル（DB） → シリアライザ（JSON化） → 応　、という一　　。
- … 4つのクラス（一　GET・新規POST・更新PUT・削除DELETE）で人員のCRUDを実現。全クラス共　で「　セッションで　　→　対象取　→　シリアライザで　　・検査 →　　切なステータスコードで応　」というパターンを　む。
- … `ModelSerializer` + `fields = '__all__'` で、データ⇔JSONの　　を数行で実現。　し　（`data=` だけ／　1引数つき）で新規・更新を切り　える。
- … URLとビューを結ぶ　線　。`path('　　', 　　.as_view(), name='　　')` の3素。`<int:pk>` でURLから　を受け取る。　じ `backup` 関数を多数のURLが共　し、`name` で処　を　岐。
- … 1ページの件数を管　者設　（`menu_row`）から　　決める。応　に `count`・`page_size`・`results` を含める。
- … `pre_save` / `post_save` / `post_delete` で、　存・削除のたびに　。`get_changes` が「　わった項　だけ」を前後つきで差　計算。ビューには　　コードが

一切　いのに監査が成　する。

- … `CurrentRequestMiddleware` がリクエストをスレッドに　管し、シグナルから「　が操作したか」を取　可　にする。`apps.py` の `ready` での `import kosu.signals` がシグナルを　効化する　。
- … 重い処　を　スレッドで　走りさせ、即座にタスクIDを　す　。ブラウザは後から `check_task_status` で状　を　い合わせ、完成したら `download_file` で取　する。
- … プロジェクト全体の設　。MIDDLEWARE の順　・CORS（　オリジン許可）・REST_FRAMEWORK（既　はJSONのみ）・SESSION（DB　存）が　章の　　。
- … クッキー（　）で　人確　するのがセッション　証。その「自　」を悪　する攻撃を　ぐのが CSRF 対策。　者は　重チェックの関係。
- … 200成功 / 201作成 / 202処　　/ 204削除成功 / 400　　ミス / 401　　証 / 403権限なし / 404なし。　号で結果が一　で　かる。

これで、　アプリのサーバー側（バックエンド）の全体像と、人員管　機　の隅々まで　めるようになりました。次章では、この章で何　も　場した　　と、その　にある　　を詳しく掘り下げます。シリアライザが　　し、ビューが操作していた「データそのもの」の正体に　ります。

次章:　8章: データベースとモデル